I want an animated explainer made in the "Product Launch Reveal" visual style (Popular Explainer Looks). Below is the complete source code of a reference animation in this exact style, plus a small sound-effects kit. Use them as your style guide and rebuild it around my content.

MY CONTENT:
[Describe your topic, or paste your script here. If this is left as is, ask me for it before building.]

STORYBOARD FIRST: before building the animation, make a VISUAL storyboard and wait for my OK.
- One HTML page (show it as an artifact if you can) with a grid of panels, one per beat (about 6-10). Each panel is a canvas showing that beat's key frame drawn in this style: use the H library and the reference's own drawing, texture and finishing functions so the panels already look like the finished piece. Still frames only, no animation or sound yet.
- Under each panel: the beat number, its start and end time, the exact on-screen text, how it gets there from the previous panel (continuous: which elements carry over and how they move or change; or a scene change: which transition, and why the story wants one), and its key sounds.
- Below the grid, list any facts or numbers you had to assume.
When I approve it (or after my changes), build the full animation exactly as storyboarded, reusing the storyboard's drawing code.

WHAT TO BUILD
One self-contained HTML file (show it as an artifact if you can):
- A 1280x720 canvas, scaled to fit the window with letterboxing, and a small control bar: play/pause, a scrub slider, the current time, a sound on/off toggle, and loop on by default.
- A new `explainer(ctx, W, H_, t)` that tells MY CONTENT in this style. Aim for 12-30 seconds unless my content needs more; set the length in a DURATION constant.
- A `scoreSound(k)` function that schedules the sound effects with the SOUND kit below (see SOUND). Play it in sync from the player: on play call `stopSound = SOUND.live(scoreSound, currentT, { voiceover: false })`, and call `stopSound()` on pause, on seek and when the sound toggle is turned off.
- Include the H helper library and the SOUND kit below unchanged, and keep the reference's own helper functions where they are useful.
- Load these Google Fonts before the first draw (await document.fonts.load for each spec you use): Inter
  <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;800&display=block">
- Expose `window.seek = (t) => { /* draw frame at time t */ }`, `window.DURATION` and `window.scoreSound`, so the animation can be rendered frame by frame to video later.

HOW TO STAY IN THE STYLE
- Reuse the reference's rendering techniques, palette, textures, lettering, line quality, arrows, labels, number treatment, transitions and timing feel. Change the story, the text and the things being drawn; do not change the medium.

- Draw every subject accurately for my topic. Don't carry over the reference's props and drawing recipes (its characters, robots, cats and dogs, rings, icons) unless they genuinely fit, and double-check that the science and facts shown in the visuals are right.
- The surface texture and finishing passes are what make the style read as the real medium (paper grain, ink pooling, misregistration, grain, glow, scanlines and so on). Copy those functions from the reference and run them on every frame at the same strength. Don't simplify or skip them to save effort.
- Every frame is a pure function of t: create all randomness from H.rng(seed) / H.makeNoise(seed) inside the draw (never Math.random), so the same t always gives the same frame. Caching static textures across frames is fine.
- It is video, often watched on a phone: at 1280x720 keep headlines >= 44px, every label or caption the story depends on >= 28px (including small annotations and the corner header), key numbers >= 64px, and frame each beat so it fills most of the screen. Keep frames fast (well under 100 ms each if you can).
- In the reference, `explainer()` is the explainer (it explains "How does AI learn?"), `draw()` is the single hero frame of the style, and `explainerSound(k)` is its sound design. Study them, but do not reuse that topic unless my content asks for it.

STRUCTURE
- Structure it like a real explainer, not a title card: a hook that gets out of the way within ~2 s, then beats where something changes every 1-2 s; arrows that draw on to show cause and flow; labels that land on what they label; any full scene change done the medium's own way (see the reference); an emphasis move on the key moment; a short takeaway held ~1 s at the end.

FLOW BETWEEN BEATS
- Decide beat by beat what the story needs. A new beat does not automatically mean a new scene.
- When the next idea builds on the last, keep it continuous: elements that matter stay on screen and move, grow, re-arrange or transform into the next idea (a label becomes the next heading, a counter grows into a chart, the camera travels to a new part of the same scene, a shape morphs into the next one).
- When the story really moves on (a new topic, a jump in time or place, a before/after contrast, a reveal), a full scene change is the right call. Do it the medium's own way, as the reference does.
- Aim for a mix that fits the content, not a rule: avoid both a hard cut on every beat and forcing everything into one unbroken shot. The reference's own transitions show how this medium changes scene; its scene count is not a template to copy.

SOUND (no voiceover: a fuller mix)
- Sync is everything: a sound starts on the exact frame its event appears, using the same times as the picture.
- Score the events that land: things popping or stamping on, labels landing, arrows completing, scene transitions, a wrong/right moment, the key number (the biggest hit) and the takeaway landing (a resolving sound).
- Drawing makes its medium's noise while it draws (marker, chalk, pen strokes matched to the draw-on length; typing while text types; chip blips while game dialog prints). Counters tick at most ~12 times a second, slowing as they settle.

- A quiet ambience bed suited to the medium is welcome, and music only if it suits the style (k.music, gain 0.6 or less).
- Keep a hierarchy: effects in front, any bed or music clearly underneath, no more than ~3 sounds at once, and small silences so the big hits read. Medium-true palette, not generic swooshes.

If you are Claude Code with a shell: render an MP4 by loading the file in headless Chrome (e.g. puppeteer), calling window.seek(i/30) for each frame, capturing the canvas and piping the frames into ffmpeg. Render the effects to a WAV with `SOUND.renderOffline(window.scoreSound, DURATION, 48000, { voiceover: false })` (it returns interleaved stereo floats) and mux it in.

STYLE CARD
Name: Product Launch Reveal
Family: Popular Explainer Looks
Look: A big-tech keynote product film: one glossy hero object on a near-black stage, lit like a studio shot with soft key, rim light, a sweeping specular highlight and a floor reflection, huge tight-tracked sans headlines, gradient key numbers with a soft glow and hairline spec callouts.
Motion: Rim light finds the object in the dark, the key light fades up as the camera slowly orbits and pushes in, a highlight glides across the glass, the edge ring ignites, then the headline arrives one word at a time and hairline callouts draw out to their spec labels.
How it explains: Told as a keynote launch film around one hero object (the model, a glass-topped chip): one continuous studio camera that orbits and pushes between beats, hairline callouts and arrows for flow, a clean red strike for the wrong guess while the chip's inner layers light up backwards to adjust, a giant violet-to-orange 97% with a glow bloom and gleam, and a light-sweep wipe to a pristine white stage for the tagline.

H HELPER LIBRARY (lib/harness.js; the registerStyle/H.render parts are only needed by the original gallery)
```js
// Shared helpers + style registry for the style-frame explorations.
// Every style is a pure function of time: draw(ctx, W, H, t). Same t => same pixels.
(function () {
  const STYLES = (window.STYLES = window.STYLES || []);
  window.registerStyle = (s) => STYLES.push(s);

  const H = (window.H = {});

  // ---------- seeded randomness ----------
  // H.rng(seed) -> r() in [0,1). Extras: r.range(a,b), r.int(a,b), r.pick(arr), r.gauss(), r.chance(p)
  H.rng = function (seed = 1) {
    let a = typeof seed === 'string' ? hashStr(seed) : (seed >>> 0) || 1;
    const r = function () {
      a |= 0; a = (a + 0x6d2b79f5) | 0;
      let t = Math.imul(a ^ (a >>> 15), 1 | a);
      t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
```

```
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
  r.range = (lo, hi) => lo + r() * (hi - lo);
  r.int = (lo, hi) => Math.floor(lo + r() * (hi - lo + 1));
  r.pick = (arr) => arr[Math.floor(r() * arr.length)];
  r.chance = (p) => r() < p;
  r.gauss = () => { let u = 0, v = 0; while (!u) u = r(); while (!v) v = r(); return Math.sqrt(-2 *
Math.log(u)) * Math.cos(2 * Math.PI * v); };
  return r;
 };
 function hashStr(s) { let h = 2166136261; for (let i = 0; i < s.length; i++) { h ^=
s.charCodeAt(i); h = Math.imul(h, 16777619); } return h >>> 0; }

 // ---------- gradient noise ----------
 // H.makeNoise(seed) -> { n2(x,y), n3(x,y,z), fbm2(x,y,oct), fbm3(x,y,z,oct) }  (outputs
roughly -1..1)
 H.makeNoise = function (seed = 1) {
  const r = H.rng(seed), p = new Uint8Array(512), perm = [...Array(256).keys()];
  for (let i = 255; i > 0; i--) { const j = Math.floor(r() * (i + 1)); [perm[i], perm[j]] = [perm[j],
perm[i]]; }
  for (let i = 0; i < 512; i++) p[i] = perm[i & 255];
  const fade = (t) => t * t * t * (t * (t * 6 - 15) + 10);
  const lerp = (a, b, t) => a + (b - a) * t;
  const g3 = (h, x, y, z) => { h &= 15; const u = h < 8 ? x : y, v = h < 4 ? y : h === 12 || h ===
14 ? x : z; return ((h & 1) ? -u : u) + ((h & 2) ? -v : v); };
  function n3(x, y, z) {
   const X = Math.floor(x) & 255, Y = Math.floor(y) & 255, Z = Math.floor(z) & 255;
   x -= Math.floor(x); y -= Math.floor(y); z -= Math.floor(z);
   const u = fade(x), v = fade(y), w = fade(z);
   const A = p[X] + Y, AA = p[A] + Z, AB = p[A + 1] + Z, B = p[X + 1] + Y, BA = p[B] + Z, BB
= p[B + 1] + Z;
   return lerp(lerp(lerp(g3(p[AA], x, y, z), g3(p[BA], x - 1, y, z), u), lerp(g3(p[AB], x, y - 1, z),
g3(p[BB], x - 1, y - 1, z), u), v),
     lerp(lerp(g3(p[AA + 1], x, y, z - 1), g3(p[BA + 1], x - 1, y, z - 1), u), lerp(g3(p[AB + 1], x, y
- 1, z - 1), g3(p[BB + 1], x - 1, y - 1, z - 1), u), v), w);
  }
  const n2 = (x, y) => n3(x, y, 0.5);
  const fbm3 = (x, y, z, oct = 4) => { let s = 0, a = 0.5, f = 1, norm = 0; for (let i = 0; i < oct;
i++) { s += a * n3(x * f, y * f, z * f); norm += a; a *= 0.5; f *= 2; } return s / norm; };
  const fbm2 = (x, y, oct = 4) => fbm3(x, y, 0.5, oct);
  return { n2, n3, fbm2, fbm3 };
 };

 // ---------- math ----------
 H.lerp = (a, b, t) => a + (b - a) * t;
 H.clamp = (v, lo = 0, hi = 1) => Math.max(lo, Math.min(hi, v));
 H.map = (v, a, b, c, d) => c + ((v - a) / (b - a)) * (d - c);
 H.smooth = (e0, e1, x) => { const t = H.clamp((x - e0) / (e1 - e0)); return t * t * (3 - 2 * t); };
```

```
  H.ease = {
    inOut: (t) => (t < 0.5 ? 4 * t * t * t : 1 - Math.pow(-2 * t + 2, 3) / 2),
    out: (t) => 1 - Math.pow(1 - t, 3),
    in: (t) => t * t * t,
    back: (t) => { const c = 1.70158; return 1 + (c + 1) * Math.pow(t - 1, 3) + c * Math.pow(t - 1,
2); },
  };

  // ---------- color ----------
  H.hex = (h) => { h = h.replace('#', ''); if (h.length === 3) h = h.split('').map((c) => c +
c).join(''); const n = parseInt(h, 16); return [(n >> 16) & 255, (n >> 8) & 255, n & 255]; };
  H.rgba = (hexOrArr, a = 1) => { const [r, g, b] = Array.isArray(hexOrArr) ? hexOrArr :
H.hex(hexOrArr); return `rgba(${r | 0},${g | 0},${b | 0},${a})`; };
  H.mix = (c1, c2, t) => { const a = Array.isArray(c1) ? c1 : H.hex(c1), b = Array.isArray(c2) ?
c2 : H.hex(c2); return [0, 1, 2].map((i) => a[i] + (b[i] - a[i]) * t); };

  // ---------- canvas utilities ----------
  // Offscreen canvas of the same logical size (use for layers, masks, compositing).
  H.layer = (W, H_, ) => { const c = document.createElement('canvas'); c.width = W; c.height
= H_; return c; };
  // Monochrome grain / film noise over the whole canvas. amount ~ 0.03-0.15
  H.grain = function (ctx, W, Hh, amount = 0.08, seed = 7, mono = true) {
    const img = ctx.getImageData(0, 0, ctx.canvas.width, ctx.canvas.height), d = img.data, r =
H.rng(seed), k = amount * 255;
    for (let i = 0; i < d.length; i += 4) {
      const n = (r() - 0.5) * k;
      if (mono) { d[i] += n; d[i + 1] += n; d[i + 2] += n; }
      else { d[i] += (r() - 0.5) * k; d[i + 1] += (r() - 0.5) * k; d[i + 2] += (r() - 0.5) * k; }
    }
    ctx.putImageData(img, 0, 0);
  };
  // Paper-ish texture tile: fills with base color + fibrous noise (cheap; drawn into ctx).
  H.paper = function (ctx, W, Hh, base = '#f3ecdc', seed = 3, strength = 0.06) {
    ctx.save(); ctx.fillStyle = base; ctx.fillRect(0, 0, W, Hh);
    const nz = H.makeNoise(seed), cell = 4, [br, bg, bb] = H.hex(base);
    for (let y = 0; y < Hh; y += cell) for (let x = 0; x < W; x += cell) {
      const v = nz.fbm2(x * 0.01, y * 0.01, 3) * 0.6 + nz.n2(x * 0.15, y * 0.04) * 0.4;
      const s = 1 + v * strength * 2;
      ctx.fillStyle = `rgb(${br * s | 0},${bg * s | 0},${bb * s | 0})`; ctx.fillRect(x, y, cell, cell);
    }
    ctx.restore();
  };
  // Vignette
  H.vignette = function (ctx, W, Hh, strength = 0.5, color = '0,0,0') {
    const g = ctx.createRadialGradient(W / 2, Hh / 2, Math.min(W, Hh) * 0.3, W / 2, Hh / 2,
Math.hypot(W, Hh) * 0.6);
    g.addColorStop(0, `rgba(${color},0)`); g.addColorStop(1, `rgba(${color},${strength})`);
    ctx.save(); ctx.fillStyle = g; ctx.fillRect(0, 0, W, Hh); ctx.restore();
```

```
  };

  // ---------- rendering entry ----------
  // Renders style into canvas at logical 1280x720 (canvas can be any pixel size; it is scaled).
  H.W = 1280; H.H = 720;
  H.loadFonts = async function (style) {
    const specs = style.fonts || [];
    try { const sample = 'AaZz09' + (window.FONT_SAMPLE || '');
      await Promise.all(specs.map((f) => document.fonts.load(f, sample))); await
document.fonts.ready; } catch (e) { /* render anyway */ }
  };
  H.render = function (style, canvas, t) {
    const ctx = canvas.getContext('2d', { willReadFrequently: !!style.readsPixels });
    const sx = canvas.width / H.W, sy = canvas.height / H.H;
    ctx.setTransform(1, 0, 0, 1, 0, 0); ctx.globalAlpha = 1; ctx.globalCompositeOperation =
'source-over'; ctx.filter = 'none';
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.save(); ctx.scale(sx, sy);
    style.draw(ctx, H.W, H.H, t == null ? style.frameT || 0 : t);
    ctx.restore();
  };
})();
```

SOUND KIT (lib/sound.js)
```js
// sound.js: a small synthesized sound kit for the style explainers. Plain Web Audio, no audio
files.
// Voiceover mode: SOUND.kit(ctx, { voiceover: true }) drops beds and music, plays anything
marked minor
// (play(..., { minor: true }), tone/noise { minor: true }, or scheduled inside k.withMinor(() =>
...)) at about 60%
// volume instead of full, and sits all effects a little lower, so the narration stays on top while
the piece still
// sounds alive. In the full mix, minor sounds play at full volume. Scores can read k.vo to
branch further.
// A style adds   explainerSound(k) { k.bed('room', 0, D); k.play('pop', 1.25); ... }   and the
same
// call schedules into a live AudioContext (the player) or an OfflineAudioContext (the video
render).
// Times are seconds on the explainer's own clock. Randomness is seeded, so every render
is identical.
(function () {
  'use strict';
  const SOUND = (window.SOUND = {});
  const TINY = 1e-4;

  // seeded random in [0,1), keyed by any string
```

```
  function rand(key) {
    let h = 2166136261;
    for (let i = 0; i < key.length; i++) { h ^= key.charCodeAt(i); h = Math.imul(h, 16777619); }
    let a = h >>> 0 || 1;
    return () => { a = (a + 0x6d2b79f5) | 0; let t = Math.imul(a ^ (a >>> 15), 1 | a); t = (t +
Math.imul(t ^ (t >>> 7), 61 | t)) ^ t; return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };
  }

  // note names to Hz: 'A4' = 440, 'C#5', 'Eb3'
  const STEP = { C: 0, D: 2, E: 4, F: 5, G: 7, A: 9, B: 11 };
  function hz(n) {
    if (typeof n === 'number') return n;
    const m = /^([A-G])([#b]?)(-?\d)$/.exec(n);
    const semis = STEP[m[1]] + (m[2] === '#' ? 1 : m[2] === 'b' ? -1 : 0) + (Number(m[3]) - 4) *
12 - 9;
    return 440 * Math.pow(2, semis / 12);
  }
  SOUND.hz = hz;

  SOUND.kit = function (ctx, opts) {
    const o = opts || {};
    const VO = !!o.voiceover;
    const MINOR = 0.6; // voiceover-mode volume for minor sounds
    let inMinor = false;
    const gainOf = (p, d) => (p.gain == null ? d : p.gain) * (VO && (p.minor || inMinor) ?
MINOR : 1);
    const T0 = (o.start || 0) - 0; // explainer time that maps to ctx time `base`
    const base = ctx.currentTime + (o.delay || 0);
    // Start times snap to the sample grid (+ a quarter sample): in Chrome a source started a
hair off the grid can
    // spike to about twice its set level.
    const SR = ctx.sampleRate;
    const at = (t) => (Math.round((base + (t - T0)) * SR) + 0.25) / SR;

    // ---- buses: sfx, amb, music -> compressor -> master; one shared reverb send
    const master = ctx.createGain(); master.gain.value = o.master == null ? 0.9 : o.master;
    const comp = ctx.createDynamicsCompressor();
    comp.threshold.value = -16; comp.knee.value = 8; comp.ratio.value = 3;
comp.attack.value = 0.004; comp.release.value = 0.18;
    comp.connect(master); master.connect(o.dest || ctx.destination);
    const bus = {};
    for (const [name, g] of Object.entries({ sfx: VO ? 0.7 : 0.85, amb: 0.5, music: 0.45 })) {
bus[name] = ctx.createGain(); bus[name].gain.value = g; bus[name].connect(comp); }

    // generated room reverb: decaying stereo noise, darker toward the tail
    const verb = ctx.createConvolver();
    {
```

```
    const secs = 1.6, n = Math.floor(secs * ctx.sampleRate), ir = ctx.createBuffer(2, n,
ctx.sampleRate);
    for (let c = 0; c < 2; c++) {
      const r = rand('verb' + c), d = ir.getChannelData(c); let lp = 0;
      for (let i = 0; i < n; i++) { const u = i / n; lp += (0.6 - 0.45 * u) * ((r() * 2 - 1) - lp); d[i] = lp *
Math.pow(1 - u, 3); }
    }
    verb.buffer = ir;
  }
  const verbIn = ctx.createGain(); verbIn.gain.value = 0.9; verbIn.connect(verb);
verb.connect(comp);

  // seeded noise buffers: white, pink-ish and brown
  const noiseBuf = {};
  {
    const n = Math.floor(2.5 * ctx.sampleRate);
    for (const kind of ['white', 'pink', 'brown']) {
      const b = ctx.createBuffer(1, n, ctx.sampleRate), d = b.getChannelData(0), r =
rand('noise-' + kind);
      let p0 = 0, p1 = 0, p2 = 0, br = 0;
      for (let i = 0; i < n; i++) {
        const w = r() * 2 - 1;
        if (kind === 'white') d[i] = w;
        else if (kind === 'pink') { p0 = 0.997 * p0 + 0.029 * w; p1 = 0.985 * p1 + 0.032 * w; p2 =
0.95 * p2 + 0.048 * w; d[i] = (p0 + p1 + p2 + w * 0.05) * 2.2; }
        else { br = (br + 0.02 * w) / 1.02; d[i] = br * 3.2; }
      }
      noiseBuf[kind] = b;
    }
  }

  // ---- automation helpers
  // a value or [[dt, value], ...] from the voice start; ramps are exponential (frequencies)
  function sweep(param, v, t) {
    if (!Array.isArray(v)) { param.setValueAtTime(v, t); return; }
    param.setValueAtTime(v[0][1], t + v[0][0]);
    for (let i = 1; i < v.length; i++) param.exponentialRampToValueAtTime(Math.max(v[i][1],
1), t + v[i][0]);
  }
  // amplitude: linear attack to `gain`, optional hold, exponential release to silence by `dur`
  function shape(param, t, gain, dur, a, hold) {
    a = Math.min(a == null ? 0.004 : a, dur * 0.9);
    hold = Math.min(hold || 0, dur - a);
    param.setValueAtTime(0, t);
    param.linearRampToValueAtTime(gain, t + a);
    if (hold > 0) param.setValueAtTime(gain, t + a + hold);
    param.exponentialRampToValueAtTime(TINY, t + dur);
    param.setValueAtTime(0, t + dur + 0.001);
```

```
  }
  function out(node, p) {
    const pan = ctx.createStereoPanner(); pan.pan.value = Math.max(-1, Math.min(1, p.pan
|| 0));
    node.connect(pan); pan.connect(bus[p.bus || 'sfx']);
    if (p.verb) { const s = ctx.createGain(); s.gain.value = p.verb; pan.connect(s);
s.connect(verbIn); }
  }
  function filterNode(p, t) {
    if (!p.filter) return null;
    const f = ctx.createBiquadFilter(); f.type = p.filter; f.Q.value = p.q == null ? 1 : p.q;
    sweep(f.frequency, p.f == null ? 1000 : p.f, t);
    return f;
  }

  // ---- primitives
  // tone(t, { f, type, dur, gain, a, hold, pan, verb, bus, filter, f2 (filter freq), q, detune })
  function tone(time, p) {
    const t = at(time), dur = p.dur || 0.2;
    if (t + dur < ctx.currentTime || (time < T0 - 0.001 && dur < 1.5)) return;
    const osc = ctx.createOscillator(); osc.type = p.type || 'sine';
    sweep(osc.frequency, Array.isArray(p.f) ? p.f.map(([d, v]) => [d, hz(v)]) : hz(p.f || 440), t);
    if (p.detune) osc.detune.value = p.detune;
    const g = ctx.createGain(); shape(g.gain, t, gainOf(p, 0.3), dur, p.a, p.hold);
    let node = osc;
    if (p.filter) { const f = ctx.createBiquadFilter(); f.type = p.filter; f.Q.value = p.q == null ? 0.8
: p.q; sweep(f.frequency, p.f2 || 2000, t); osc.connect(f); node = f; }
    node.connect(g); out(g, p);
    osc.start(t); osc.stop(t + dur + 0.02);
  }
  // noise(t, { dur, color, filter, f, q, gain, a, hold, pan, verb, bus, rate })
  function noise(time, p) {
    const t = at(time), dur = p.dur || 0.2;
    if (t + dur < ctx.currentTime || (time < T0 - 0.001 && dur < 1.5)) return;
    const src = ctx.createBufferSource(); src.buffer = noiseBuf[p.color || 'white']; src.loop =
true;
    if (p.rate) src.playbackRate.value = p.rate;
    const g = ctx.createGain(); shape(g.gain, t, gainOf(p, 0.3), dur, p.a, p.hold);
    const f = filterNode(p, t);
    if (f) { src.connect(f); f.connect(g); } else src.connect(g);
    out(g, p);
    const off = rand('off' + time + (p.f || ''))() * 2;
    src.start(t, off); src.stop(t + dur + 0.02);
  }

  // ---- named sounds. play(name, t, opts): opts.gain scales the whole sound, opts.pan
places it.
  const S = {
```

```
    // UI / motion graphics
    pop: (t, o) => { tone(t, { f: [[0, o.f || 620], [0.07, (o.f || 620) * 0.45]], dur: 0.1, gain: 0.32 * o.g, pan: o.pan }); noise(t, { dur: 0.012, filter: 'highpass', f: 3000, gain: 0.12 * o.g, pan: o.pan }); },
    tick: (t, o) => noise(t, { dur: 0.018, filter: 'bandpass', f: o.f || 4200, q: 2.5, gain: 0.3 * o.g, pan: o.pan }),
    click: (t, o) => { tone(t, { f: o.f || 1900, type: 'square', dur: 0.02, gain: 0.08 * o.g, filter: 'lowpass', f2: 3000, pan: o.pan }); noise(t, { dur: 0.01, filter: 'highpass', f: 2500, gain: 0.15 * o.g, pan: o.pan }); },
    tock: (t, o) => { tone(t, { f: o.f || 820, dur: 0.07, gain: 0.25 * o.g, pan: o.pan }); noise(t, { dur: 0.05, filter: 'bandpass', f: o.f || 820, q: 9, gain: 0.35 * o.g, pan: o.pan }); },
    blip: (t, o) => tone(t, { f: o.f || 'E6', type: o.type || 'square', dur: o.dur || 0.07, gain: 0.07 * o.g, a: 0.002, hold: (o.dur || 0.07) * 0.6, pan: o.pan }),
    whoosh: (t, o) => { const d = o.dur || 0.45; noise(t, { dur: d, color: 'pink', filter: 'bandpass', f: [[0, 350], [d * 0.55, o.top || 2600], [d, 700]], q: 0.9, gain: 0.32 * o.g, a: d * 0.5, pan: o.pan, verb: 0.15 }); },
    swish: (t, o) => { const d = o.dur || 0.18; noise(t, { dur: d, filter: 'highpass', f: [[0, 1800], [d, 5200]], q: 0.7, gain: 0.22 * o.g, a: d * 0.35, pan: o.pan }); },
    riser: (t, o) => { const d = o.dur || 0.8; noise(t, { dur: d, color: 'pink', filter: 'bandpass', f: [[0, 300], [d, 4000]], q: 1.2, gain: 0.25 * o.g, a: d * 0.95, pan: o.pan, verb: 0.2 }); tone(t, { f: [[0, 180], [d, 720]], type: 'triangle', dur: d, gain: 0.06 * o.g, a: d * 0.9 }); },
    thud: (t, o) => { tone(t, { f: [[0, o.f || 130], [0.16, 42]], dur: 0.26, gain: 0.55 * o.g, pan: o.pan }); noise(t, { dur: 0.09, color: 'brown', filter: 'lowpass', f: 500, gain: 0.35 * o.g, pan: o.pan }); },
    impact: (t, o) => { S.thud(t, o); noise(t, { dur: 0.7, color: 'pink', filter: 'lowpass', f: [[0, 2400], [0.6, 300]], gain: 0.3 * o.g, verb: 0.35, pan: o.pan }); },
    stamp: (t, o) => { S.thud(t, { ...o, f: 160 }); noise(t, { dur: 0.05, filter: 'bandpass', f: 1300, q: 1.2, gain: 0.4 * o.g, pan: o.pan }); noise(t + 0.004, { dur: 0.012, filter: 'highpass', f: 3500, gain: 0.25 * o.g, pan: o.pan }); },
    ding: (t, o) => { const f = hz(o.f || 'E6'); tone(t, { f, dur: 1.3, gain: 0.2 * o.g, verb: 0.3, pan: o.pan }); tone(t, { f: f * 2.76, dur: 0.5, gain: 0.05 * o.g, verb: 0.3, pan: o.pan }); },
    success: (t, o) => { S.ding(t, { ...o, f: o.f || 'C6' }); S.ding(t + 0.09, { ...o, f: hz(o.f || 'C6') * 1.5, g: o.g * 0.9 }); },
    error: (t, o) => { tone(t, { f: o.f || 'A3', type: 'square', dur: 0.13, gain: 0.09 * o.g, hold: 0.08, filter: 'lowpass', f2: 1400, pan: o.pan }); tone(t + 0.15, { f: hz(o.f || 'A3') * 0.8, type: 'square', dur: 0.22, gain: 0.09 * o.g, hold: 0.14, filter: 'lowpass', f2: 1200, pan: o.pan }); },
    buzz: (t, o) => { const d = o.dur || 0.3; tone(t, { f: o.f || 110, type: 'sawtooth', dur: d, gain: 0.12 * o.g, hold: d * 0.7, filter: 'lowpass', f2: 900, pan: o.pan }); tone(t, { f: (o.f || 110) * 1.01, type: 'sawtooth', dur: d, gain: 0.1 * o.g, hold: d * 0.7, filter: 'lowpass', f2: 900, pan: o.pan }); },
    sparkle: (t, o) => { const r = rand('sparkle' + t), notes = ['C6', 'D6', 'E6', 'G6', 'A6', 'C7']; for (let i = 0; i < (o.n || 6); i++) { const ti = t + i * 0.04 + r() * 0.01, f = hz(notes[Math.floor(r() * notes.length)]), pan = r() * 1.2 - 0.6; tone(ti, { f, dur: 0.6, gain: 0.04 * o.g, a: 0.002, verb: 0.35, pan }); tone(ti, { f: f * 2.76, dur: 0.2, gain: 0.012 * o.g, a: 0.002, pan }); } },
    glitch: (t, o) => { const r = rand('glitch' + t); for (let i = 0; i < 7; i++) tone(t + i * 0.022, { f: 200 + r() * 1800, type: 'square', dur: 0.02, gain: 0.07 * o.g, a: 0.001, hold: 0.015, pan: r() * 1.4 - 0.7 }); noise(t, { dur: 0.16, filter: 'bandpass', f: 2500, q: 0.6, gain: 0.12 * o.g }); },
    shutter: (t, o) => { S.click(t, o); noise(t + 0.06, { dur: 0.04, filter: 'bandpass', f: 2200, q: 1, gain: 0.2 * o.g, pan: o.pan }); },
    count: (t, o) => S.tick(t, { ...o, f: o.f || 5200, g: o.g * 0.7 }),
```

```
    // hand-made media
    // stroke: a pen/marker/chalk/pencil stroke over `dur` seconds. material sets the surface sound.
    stroke: (t, o) => {
      const d = o.dur || 0.4, m = o.material || 'marker', r = rand('stroke' + m + t);
      const M = {
        marker: { f: 2900, q: 2.2, g: 0.16, grain: 0.05 },
        chalk: { f: 1700, q: 0.6, g: 0.22, grain: 0.028 },
        pencil: { f: 4600, q: 0.8, g: 0.16, grain: 0.04, filter: 'highpass' },
        pen: { f: 4200, q: 1.6, g: 0.08, grain: 0.06 },
        brush: { f: 900, q: 0.5, g: 0.1, grain: 0.09, color: 'pink' },
        sharpie: { f: 2400, q: 1.4, g: 0.2, grain: 0.045 },
      }[m];
      for (let x = 0; x < d; x += M.grain * (0.7 + r() * 0.6)) {
        const gd = M.grain * (0.8 + r() * 0.9);
        noise(t + x, { dur: gd, color: M.color, filter: M.filter || 'bandpass', f: M.f * (0.8 + r() * 0.4), q: M.q, gain: M.g * o.g * (0.6 + r() * 0.5), a: gd * 0.3, pan: (o.pan || 0) + (r() - 0.5) * 0.2 });
      }
    },
    // paper: picking up / slapping down / turning a sheet
    paper: (t, o) => { const d = o.dur || 0.35, r = rand('paper' + t); for (let x = 0; x < d; x += 0.025 + r() * 0.03) noise(t + x, { dur: 0.05, filter: 'bandpass', f: 1200 + r() * 2600, q: 0.9, gain: 0.12 * o.g * (1 - x / d * 0.5), pan: o.pan }); },
    slap: (t, o) => { noise(t, { dur: 0.07, filter: 'highpass', f: 900, gain: 0.45 * o.g, a: 0.001, pan: o.pan }); tone(t, { f: [[0, 180], [0.06, 70]], dur: 0.1, gain: 0.25 * o.g, pan: o.pan }); },
    tape: (t, o) => { noise(t, { dur: o.dur || 0.5, filter: 'bandpass', f: [[0, 900], [o.dur || 0.5, 2200]], q: 1.8, gain: 0.18 * o.g, a: 0.02, pan: o.pan }); },
    press: (t, o) => { S.thud(t, { ...o, f: 95, g: o.g * 0.9 }); noise(t + 0.01, { dur: 0.25, color: 'brown', filter: 'lowpass', f: 700, gain: 0.25 * o.g, pan: o.pan }); },
    roller: (t, o) => { const d = o.dur || 0.6; noise(t, { dur: d, color: 'brown', filter: 'bandpass', f: 380, q: 1.5, gain: 0.3 * o.g, a: 0.05, hold: d * 0.6, pan: o.pan }); for (let x = 0; x < d; x += 0.07) S.tock(t + x, { f: 300, g: o.g * 0.15, pan: o.pan }); },
    erase: (t, o) => { const d = o.dur || 0.6, r = rand('erase' + t); for (let x = 0; x < d; x += 0.06 + r() * 0.04) noise(t + x, { dur: 0.08, color: 'pink', filter: 'bandpass', f: 700 + r() * 500, q: 0.8, gain: 0.22 * o.g, a: 0.03, pan: (r() - 0.5) * 0.5 }); },

    // machines and screens
    key: (t, o) => { const r = rand('key' + t); noise(t, { dur: 0.025, filter: 'bandpass', f: 2600 + r() * 1200, q: 2, gain: 0.25 * o.g, a: 0.001, pan: o.pan }); tone(t, { f: 160 + r() * 40, dur: 0.04, gain: 0.12 * o.g, pan: o.pan }); },
    typing: (t, o) => { const r = rand('typing' + t), n = o.n || 8, rate = o.rate || 0.09; for (let i = 0; i < n; i++) S.key(t + i * rate * (0.75 + r() * 0.5), { g: o.g * (0.7 + r() * 0.4), pan: o.pan }); },
    typebar: (t, o) => { S.key(t, o); noise(t + 0.008, { dur: 0.03, filter: 'bandpass', f: 1100, q: 4, gain: 0.35 * o.g, pan: o.pan }); },
    bell: (t, o) => S.ding(t, { ...o, f: o.f || 'A6' }),
    boot: (t, o) => { tone(t, { f: [[0, 80], [0.35, 1600]], type: 'sine', dur: 0.5, gain: 0.08 * o.g, a: 0.3 }); S.blip(t + 0.45, { ...o, f: 'C6', dur: 0.12 }); },
```

```
    crtOn: (t, o) => { noise(t, { dur: 0.25, filter: 'highpass', f: 2000, gain: 0.2 * o.g, a: 0.001 });
tone(t, { f: 15700, dur: 1.5, gain: 0.02 * o.g, a: 0.1, hold: 1 }); S.thud(t, { ...o, f: 90, g: o.g * 0.5
}); },
    neon: (t, o) => { const d = o.dur || 0.25, r = rand('neon' + t); for (let x = 0; x < d; x +=
0.035 + r() * 0.04) tone(t + x, { f: 120, type: 'sawtooth', dur: 0.03, gain: 0.3 * o.g * (0.5 + r() *
0.5), filter: 'bandpass', f2: 1200, q: 1.2, pan: o.pan }); },
    // chip (8-bit console) sounds: raw square/triangle, no filtering
    coin: (t, o) => { tone(t, { f: 'B5', type: 'square', dur: 0.07, gain: 0.07 * o.g, a: 0.001, hold:
0.06 }); tone(t + 0.07, { f: 'E6', type: 'square', dur: 0.3, gain: 0.07 * o.g, a: 0.001, hold: 0.1 }); },
    jump: (t, o) => tone(t, { f: [[0, 260], [0.16, 820]], type: 'square', dur: 0.18, gain: 0.07 * o.g,
a: 0.001, hold: 0.12 }),
    hurt: (t, o) => { tone(t, { f: [[0, 600], [0.25, 90]], type: 'square', dur: 0.28, gain: 0.08 * o.g,
a: 0.001, hold: 0.2 }); noise(t, { dur: 0.2, filter: 'lowpass', f: 1800, gain: 0.15 * o.g, rate: 0.25 });
},
    powerup: (t, o) => ['C5', 'E5', 'G5', 'C6', 'E6', 'G6', 'C7'].forEach((n, i) => tone(t + i * 0.045,
{ f: n, type: 'square', dur: 0.06, gain: 0.06 * o.g, a: 0.001, hold: 0.05 })),
    select: (t, o) => { S.blip(t, { ...o, f: 'A5', dur: 0.04 }); S.blip(t + 0.05, { ...o, f: 'E6', dur: 0.05
}); },
    text: (t, o) => { const n = o.n || 10, rate = o.rate || 0.05; for (let i = 0; i < n; i++) tone(t + i *
rate, { f: i % 2 ? 'C6' : 'G5', type: 'square', dur: 0.025, gain: 0.04 * o.g, a: 0.001, hold: 0.015 });
},
  };

  function play(name, t, opts) {
    const fn = S[name];
    if (!fn) throw new Error('unknown sound: ' + name);
    const o2 = Object.assign({}, opts); o2.g = o2.gain == null ? 1 : o2.gain;
    if (opts && opts.minor) withMinor(() => fn(t, o2)); else fn(t, o2);
  }

  // everything scheduled inside fn counts as minor (quieter in voiceover mode)
  function withMinor(fn) { const was = inMinor; inMinor = true; try { fn(); } finally { inMinor =
was; } }

  // ---- beds: continuous sound under a stretch of time. bed(name, t0, t1, opts)
  function bed(name, t0, t1, p) {
    if (VO) return;
    p = p || {}; const g = p.gain == null ? 1 : p.gain, d = t1 - t0, fade = Math.min(0.4, d / 4);
    const long = (q) => Object.assign({ dur: d, a: fade, hold: d - fade * 2, bus: 'amb' }, q);
    if (name === 'room') noise(t0, long({ color: 'brown', filter: 'lowpass', f: 320, gain: 0.12 * g
}));
    else if (name === 'hiss') noise(t0, long({ color: 'pink', filter: 'highpass', f: 5000, gain: 0.02 *
g }));
    else if (name === 'hum') { tone(t0, long({ f: 60, gain: 0.05 * g })); tone(t0, long({ f: 120,
gain: 0.025 * g })); tone(t0, long({ f: 180, type: 'triangle', gain: 0.012 * g })); }
    else if (name === 'rain') { noise(t0, long({ color: 'pink', filter: 'bandpass', f: 2500, q: 0.4,
gain: 0.08 * g })); const r = rand('rain'); for (let x = 0; x < d; x += 0.04 + r() * 0.08) noise(t0 + x,
{ dur: 0.01, filter: 'highpass', f: 4000, gain: 0.04 * g * r(), bus: 'amb', pan: r() * 1.6 - 0.8 }); }
```

```
    else if (name === 'projector') { noise(t0, long({ color: 'pink', filter: 'highpass', f: 3000, gain: 0.03 * g })); for (let x = 0; x < d; x += 1 / 24) S.tock(t0 + x, { f: 700, g: 0.04 * g, pan: 0.3 }); }
    else if (name === 'vinyl') { noise(t0, long({ color: 'pink', filter: 'bandpass', f: 1500, q: 0.5, gain: 0.025 * g })); const r = rand('vinyl'); for (let x = 0; x < d; x += 0.08 + r() * 0.4) noise(t0 + x, { dur: 0.006, filter: 'highpass', f: 2500, gain: 0.08 * g * r(), bus: 'amb' }); }
    else throw new Error('unknown bed: ' + name);
  }

  // ---- music: optional light beds that sit under the effects. music(name, t0, t1, { bpm, chords, gain })
  // chords: list of note arrays, one per bar, cycled. e.g. [['C4','E4','G4','B4'], ['A3','C4','E4','G4']]
  function music(name, t0, t1, p) {
    if (VO) return;
    p = p || {}; const g = p.gain == null ? 1 : p.gain, bpm = p.bpm || 100, beat = 60 / bpm, bar = beat * 4;
    const chords = p.chords || [['C4', 'E4', 'G4', 'B4'], ['A3', 'C4', 'E4', 'G4'], ['F3', 'A3', 'C4', 'E4'], ['G3', 'B3', 'D4', 'F4']];
    let i = 0;
    for (let t = t0; t < t1 - 0.05; t += bar, i++) {
      const ch = chords[i % chords.length], len = Math.min(bar, t1 - t);
      if (name === 'pad' || name === 'pulse') ch.forEach((n, k) => { tone(t, { f: n, type: 'triangle', dur: len + 0.3, gain: 0.035 * g, a: 0.25, hold: len - 0.3, bus: 'music', filter: 'lowpass', f2: 1400, pan: (k - 1.5) * 0.25, verb: 0.25 }); tone(t, { f: hz(n) * 1.004, type: 'sine', dur: len + 0.3, gain: 0.03 * g, a: 0.3, hold: len - 0.3, bus: 'music' }); });
      if (name === 'pulse') for (let b = 0; b < 4 && t + b * beat < t1 - 0.05; b++) {
        const tb = t + b * beat;
        tone(tb, { f: [[0, 110], [0.1, 50]], dur: 0.18, gain: (b % 2 ? 0.16 : 0.26) * g, bus: 'music' });
        noise(tb + beat / 2, { dur: 0.04, filter: 'highpass', f: 7000, gain: 0.05 * g, bus: 'music' });
        tone(tb + beat / 2, { f: hz(ch[0]) / 2, type: 'triangle', dur: beat * 0.45, gain: 0.08 * g, bus: 'music' });
      }
      if (name === 'chip') for (let s = 0; s < 8 && t + s * beat / 2 < t1 - 0.05; s++) {
        const ts = t + s * beat / 2;
        tone(ts, { f: hz(ch[0]) / 2, type: 'triangle', dur: beat * 0.45, gain: 0.12 * g, a: 0.001, hold: beat * 0.35, bus: 'music' });
        tone(ts, { f: ch[[0, 1, 2, 3, 2, 1, 2, 3][s]], type: 'square', dur: beat * 0.4, gain: 0.03 * g, a: 0.001, hold: beat * 0.25, bus: 'music' });
      }
    }
  }

  return { play, bed, music, tone, noise, withMinor, hz, vo: VO, sounds: Object.keys(S) };
};

// Live playback for a player: SOUND.live(scoreFn, fromT, { voiceover, master }) starts the score from explainer time
```

```
  // fromT right now and returns stop(). Call stop() on pause or seek, and SOUND.live again
on play.
  SOUND.live = function (fn, fromT, opts) {
    const ctx = new (window.AudioContext || window.webkitAudioContext)();
    fn(SOUND.kit(ctx, Object.assign({}, opts, { start: fromT || 0, delay: 0.03 })));
    return () => { try { ctx.close(); } catch (e) {} };
  };

  // Render a style's explainerSound into an OfflineAudioContext; returns interleaved float
stereo.
  SOUND.renderOffline = async function (fn, duration, sampleRate, kitOpts) {
    const sr = sampleRate || 48000, len = Math.ceil(duration * sr);
    const ctx = new OfflineAudioContext(2, len, sr);
    fn(SOUND.kit(ctx, kitOpts || {}));
    const buf = await ctx.startRendering();
    const L = buf.getChannelData(0), R = buf.getChannelData(1), inter = new
Float32Array(len * 2);
    let peak = 0;
    for (let i = 0; i < len; i++) { inter[2 * i] = L[i]; inter[2 * i + 1] = R[i]; peak = Math.max(peak,
Math.abs(L[i]), Math.abs(R[i])); }
    return { inter, peak, frames: len, sr };
  };
})();
```

REFERENCE STYLE SOURCE (styles/product-launch.js; W/H_ are always 1280x720)
```js
{
  // ---------- Product Launch Reveal: big-tech keynote film ----------
  // A single glossy hero object (an invented "model" chip-slab) on a near-black stage, lit like
a studio product
  // shot: soft key, rim light, a specular sweep, a floor reflection. Huge tight-tracked Inter,
words revealed one at
  // a time, gradient key numbers with a soft glow, hairline callouts with small spec labels,
slow eased camera moves.
  const PI = Math.PI;
  const sm = (a, b, x) => H.smooth(a, b, x);
  const io = (x) => H.ease.inOut(H.clamp(x));
  const eo = (x) => H.ease.out(H.clamp(x));
  const GV = ['#b69cff', '#e879f9', '#ff9a4d'];   // violet -> orange
  const GT = ['#5eead4', '#38bdf8', '#6d7cff'];   // teal -> blue
  const GG = ['#7cf0b8', '#2dd4bf'];              // green -> teal
  const RED = '#ff453a';
  const INK = '#f5f5f7', GREY = '#a1a1a6', DIM = '#6e6e73';

  // ---------- geometry: a rounded slab (x-z plane), with an inset die carrying a 4-layer network
----------
```

```
  const HALF = 1, RAD = 0.27, THICK = 0.12, SEG = 48; // fine corner segments so the machined side wall shades smoothly
  function rr(a, r, seg) {
    const pts = [], cs = [[a - r, a - r], [-(a - r), a - r], [-(a - r), -(a - r)], [a - r, -(a - r)]];
    cs.forEach(([cx, cz], k) => { for (let i = 0; i <= seg; i++) { const an = k * PI / 2 + (i / seg) * PI / 2; pts.push([cx + r * Math.cos(an), cz + r * Math.sin(an), an]); } });
    return pts;
  }
  const OUT = rr(HALF, RAD, SEG), DIE = rr(0.66, 0.12, 5);
  const LX = [-0.45, -0.15, 0.15, 0.45], LN = [4, 5, 5, 3];
  const NODES = LX.map((x, li) => Array.from({ length: LN[li] }, (_, k) => [x, -0.42 + (0.84 * k) / (LN[li] - 1)]));
  const EDGES = [];
  { const ra = H.rng(11), rb = H.rng(29);
    for (let li = 0; li < 3; li++) for (let a = 0; a < LN[li]; a++) for (let b = 0; b < LN[li + 1]; b++) EDGES.push({ li, a, b, wa: Math.pow(ra(), 1.6), wb: Math.pow(rb(), 1.6) }); }

  // camera: c = {cx, cy (screen point of the floor under the object), s (zoom), yaw, pitch}
  function cam(c) {
    const cyw = Math.cos(c.yaw), syw = Math.sin(c.yaw), cp = Math.cos(c.pitch), sp = Math.sin(c.pitch), D = 7, F = c.s * 1900;
    const P = (x, y, z) => { const X = x * cyw - z * syw, Z = x * syw + z * cyw, up = y * cp - Z * sp, dep = Z * cp + y * sp, k = F / (D - dep); return [c.cx + X * k, c.cy - up * k, dep]; };
    const N = (nx, nz) => [nx * cyw - nz * syw, nx * syw + nz * cyw];
    return Object.assign({}, c, { P, N, unit: F / D });
  }
  function keyed(keys, t) {
    if (t <= keys[0].t) return Object.assign({}, keys[0]);
    for (let i = 0; i < keys.length - 1; i++) {
      const a = keys[i], b = keys[i + 1];
      if (t <= b.t) { const e = io((t - a.t) / (b.t - a.t)), o = {}; for (const k in a) o[k] = H.lerp(a[k], b[k], e); return o; }
    }
    return Object.assign({}, keys[keys.length - 1]);
  }
  const poly = (x, pts) => { x.beginPath(); pts.forEach((p, i) => (i ? x.lineTo(p[0], p[1]) : x.moveTo(p[0], p[1]))); x.closePath(); };
  const gauss = (d, k) => Math.exp(-d * d * k);

  // ---------- stage ----------
  function stage(x, W, Hh, c, L, t) {
    if (L.white) {
      x.fillStyle = '#e4e4e9'; x.fillRect(0, 0, W, Hh);
      let g = x.createRadialGradient(c.cx, c.cy - 220, 40, c.cx, c.cy - 120, 900);
      g.addColorStop(0, '#ffffff'); g.addColorStop(0.55, '#f4f4f7'); g.addColorStop(1, '#dcdce2');
      x.fillStyle = g; x.fillRect(0, 0, W, Hh);
      return;
    }
```

```
    x.fillStyle = '#030304'; x.fillRect(0, 0, W, Hh);
    const k = L.key;
    let g = x.createRadialGradient(c.cx, c.cy - c.unit * 0.9, 10, c.cx, c.cy - c.unit * 0.6, 820);
    g.addColorStop(0, `rgba(44,47,60,${0.9 * k})`); g.addColorStop(0.45, `rgba(20,21,28,${0.8 * k})`); g.addColorStop(1, 'rgba(3,3,4,0)');
    x.fillStyle = g; x.fillRect(0, 0, W, Hh);
    // floor pool of light
    x.save(); x.translate(c.cx, c.cy); x.scale(1, 0.24);
    g = x.createRadialGradient(0, 0, 0, 0, 0, c.unit * 2.6);
    g.addColorStop(0, `rgba(70,74,92,${0.42 * k})`); g.addColorStop(1, 'rgba(70,74,92,0)');
    x.fillStyle = g; x.beginPath(); x.arc(0, 0, c.unit * 2.6, 0, 7); x.fill(); x.restore();
    // out-of-focus background bokeh (shallow depth of field)
    const r = H.rng(404);
    for (let i = 0; i < 6; i++) {
      const bx = r() * W + Math.sin(t * 0.2 + i) * 14 - (c.cx - 640) * 0.15, by = r.range(0.05, 0.5) * Hh, rad = r.range(40, 110), hue = r.pick(['120,110,255', '255,140,90', '90,200,255']);
      const a = r.range(0.025, 0.05) * k;
      g = x.createRadialGradient(bx, by, 0, bx, by, rad);
      g.addColorStop(0, `rgba(${hue},${a})`); g.addColorStop(0.6, `rgba(${hue},${a * 0.6})`); g.addColorStop(1, `rgba(${hue},0)`);
      x.fillStyle = g; x.beginPath(); x.arc(bx, by, rad, 0, 7); x.fill();
    }
  }

  // ---------- the hero object ----------
  function body(x, c, L, mirror) {
    const hv = L.hover, yT = hv + THICK / 2, yB = hv - THICK / 2, sc = c.unit / 271;
    const P = mirror ? (a, y, b) => c.P(a, -y, b) : c.P;
    const n = OUT.length, top = OUT.map((p) => P(p[0], yT, p[1])), bot = OUT.map((p) => P(p[0], yB, p[1]));
    const white = L.white ? 1 : 0;
    const segs = [];
    for (let i = 0; i < n; i++) {
      const j = (i + 1) % n, a1 = OUT[i][2], a2 = OUT[j][2];
      let nx = Math.cos(a1) + Math.cos(a2), nz = Math.sin(a1) + Math.sin(a2); const nl = Math.hypot(nx, nz) || 1; nx /= nl; nz /= nl;
      const [NX, NZ] = c.N(nx, nz);
      segs.push({ i, j, NX, NZ });
      if (NZ < -0.03) continue;
      // softbox key: a broad soft specular lobe (exp 18) plus a wide sheen, peak clamped so it never blows out to a bar
      const lam = Math.max(0, NX * -0.6 + NZ * 0.8), sd = Math.max(0, NX * -0.25 + NZ * 0.97), spec = Math.pow(sd, 18), sheen = Math.pow(sd, 4);
      const rim = Math.pow(Math.max(0, NX), 3);
      let v = 14 + lam * 62 * L.key + Math.min(1, spec * 0.85 + sheen * 0.3) * 140 * L.key + rim * 120 * L.rim + white * 40;
      if (mirror) v *= 0.9;
```

```
      const cc = (m) => `rgb(${Math.min(255, v * 0.96 * m) | 0},${Math.min(255, v * 0.98 * m) | 0},${Math.min(255, (v * 1.05 + 3) * m) | 0})`;
      // long straight faces are split into short pieces so the vertical profile follows the perspective-sloped edges
      const m = Math.max(1, Math.ceil(Math.hypot(top[j][0] - top[i][0], top[j][1] - top[i][1]) / 40));
      const lp = (A, B, f) => [A[0] + (B[0] - A[0]) * f, A[1] + (B[1] - A[1]) * f];
      for (let q = 0; q < m; q++) {
        const t0 = lp(top[i], top[j], q / m), t1 = lp(top[i], top[j], (q + 1) / m), b0 = lp(bot[i], bot[j], q / m), b1 = lp(bot[i], bot[j], (q + 1) / m);
        // gradient axis perpendicular to the top edge, so the bands run parallel to the edge (no sawtooth)
        let ex = t1[0] - t0[0], ey = t1[1] - t0[1]; const el = Math.hypot(ex, ey) || 1; ex /= el; ey /= el;
        const tm = [(t0[0] + t1[0]) / 2, (t0[1] + t1[1]) / 2], bm = [(b0[0] + b1[0]) / 2, (b0[1] + b1[1]) / 2];
        let gx = -ey, gy = ex, hh = (bm[0] - tm[0]) * gx + (bm[1] - tm[1]) * gy;
        if (Math.abs(hh) < 0.5) { gx = bm[0] - tm[0]; gy = bm[1] - tm[1]; hh = 1; }
        const sg = x.createLinearGradient(tm[0], tm[1], tm[0] + gx * hh, tm[1] + gy * hh);
        // polished metal: bright softbox reflection at the top, the dark studio horizon across the middle, a soft floor bounce low
        if (mirror) { sg.addColorStop(0, cc(0.5)); sg.addColorStop(0.45, cc(1)); sg.addColorStop(1, cc(1.2)); }
        else { sg.addColorStop(0, cc(1.32)); sg.addColorStop(0.3, cc(1.05)); sg.addColorStop(0.52, cc(0.62)); sg.addColorStop(0.74, cc(0.82)); sg.addColorStop(1, cc(0.42)); }
        x.fillStyle = sg; x.strokeStyle = sg; x.lineWidth = 0.9;
        poly(x, [t1, t0, b0, b1]); x.fill(); x.stroke();
      }
    }
    // bottom crease
    x.lineWidth = 1.2 * sc; x.strokeStyle = 'rgba(0,0,0,0.6)';
    for (const s of segs) if (s.NZ > 0) { x.beginPath(); x.moveTo(bot[s.i][0], bot[s.i][1]); x.lineTo(bot[s.j][0], bot[s.j][1]); x.stroke(); }
    if (mirror) { x.fillStyle = '#060608'; poly(x, bot); x.fill(); return; }

    // top: dark glass
    let minY = 1e9, maxY = -1e9; top.forEach((p) => { minY = Math.min(minY, p[1]); maxY = Math.max(maxY, p[1]); });
    poly(x, top);
    let g = x.createLinearGradient(0, minY, 0, maxY);
    g.addColorStop(0, white ? '#2a2c33' : '#1d2028'); g.addColorStop(1, '#08090c');
    x.fillStyle = g; x.fill();
    x.save(); poly(x, top); x.clip();
    // softbox reflection (glossy)
    g = x.createLinearGradient(0, minY, 0, maxY);
    g.addColorStop(0, `rgba(255,255,255,${0.16 * L.key})`); g.addColorStop(0.35, `rgba(255,255,255,${0.035 * L.key})`); g.addColorStop(1, 'rgba(255,255,255,0)');
    x.fillStyle = g; x.fillRect(0, minY, 1280, maxY - minY);
    // fixed soft streak of the softbox on the glass
```

```
  { const A = P(-1.1, yT, 0.6), B = P(0.6, yT, -1.1); const sg = x.createLinearGradient(A[0],
A[1], B[0], B[1]);
    sg.addColorStop(0, 'rgba(255,255,255,0)'); sg.addColorStop(0.3, 'rgba(255,255,255,0)');
sg.addColorStop(0.46, `rgba(255,255,255,${0.11 * L.key})`); sg.addColorStop(0.5,
`rgba(255,255,255,${0.05 * L.key})`); sg.addColorStop(0.66, 'rgba(255,255,255,0)');
sg.addColorStop(1, 'rgba(255,255,255,0)');
    x.fillStyle = sg; x.fillRect(0, minY, 1280, maxY - minY); }
  // die
  const die = DIE.map((p) => P(p[0], yT, p[1]));
  poly(x, die); x.fillStyle = 'rgba(4,5,8,0.92)'; x.fill();
  // micro-grid
  x.lineWidth = 0.6; x.strokeStyle = `rgba(150,160,200,${0.06 + 0.05 * L.act})`;
x.beginPath();
  for (let q = -0.56; q <= 0.57; q += 0.08) { let a = P(q, yT, -0.6), b = P(q, yT, 0.6);
x.moveTo(a[0], a[1]); x.lineTo(b[0], b[1]); a = P(-0.6, yT, q); b = P(0.6, yT, q); x.moveTo(a[0],
a[1]); x.lineTo(b[0], b[1]); }
  x.stroke();
  // network
  const fp = L.fwd >= 0 ? L.fwd * 3.8 - 0.4 : -9, bp = L.back >= 0 ? (1 - L.back) * 3.8 - 0.4 :
-9;
  const np = NODES.map((ly) => ly.map(([nx, nz]) => P(nx, yT, nz)));
  x.lineCap = 'round';
  for (const e of EDGES) {
    const w = H.lerp(e.wa, e.wb, L.adj), fi = gauss(fp - (e.li + 0.5), 3) * (0.35 + 0.65 * w), bi =
gauss(bp - (e.li + 0.5), 3) * (0.35 + 0.65 * w);
    const a = 0.05 + 0.26 * w * L.act + 0.85 * Math.max(fi, bi);
    const col = bi > fi ? '255,140,80' : (fi > 0.05 ? '140,230,255' : '190,180,255');
    const A = np[e.li][e.a], B = np[e.li + 1][e.b];
    x.strokeStyle = `rgba(${col},${Math.min(1, a)})`; x.lineWidth = (0.7 + 1.3 * w) * sc;
    x.beginPath(); x.moveTo(A[0], A[1]); x.lineTo(B[0], B[1]); x.stroke();
  }
  np.forEach((ly, li) => ly.forEach((p) => {
    const fi = gauss(fp - li, 3), bi = gauss(bp - li, 3), on = Math.max(fi, bi);
    const col = bi > fi ? '255,150,90' : '170,235,255';
    const rad = (2.6 + 2.4 * on) * sc;
    if (on > 0.08) { const gg = x.createRadialGradient(p[0], p[1], 0, p[0], p[1], rad * 4);
gg.addColorStop(0, `rgba(${col},${0.5 * on})`); gg.addColorStop(1, `rgba(${col},0)`);
x.fillStyle = gg; x.beginPath(); x.arc(p[0], p[1], rad * 4, 0, 7); x.fill(); }
    x.fillStyle = `rgba(${on > 0.15 ? col : '215,215,235'},${0.35 + 0.25 * L.act + 0.4 * on})`;
x.beginPath(); x.arc(p[0], p[1], rad, 0, 7); x.fill();
  }));
  // specular sweep
  if (L.spec > -0.3 && L.spec < 1.3) {
    const A = P(-1.1, yT, 1.1), B = P(1.1, yT, -1.1), u = L.spec;
    g = x.createLinearGradient(A[0], A[1], B[0], B[1]);
    const st = (o, a) => { const q = u + o; if (q > 0 && q < 1) g.addColorStop(q,
`rgba(255,255,255,${a})`); };
```

```
      g.addColorStop(0, 'rgba(255,255,255,0)'); st(-0.16, 0); st(-0.05, 0.08); st(0, 0.42 * L.key);
st(0.02, 0.18); st(0.1, 0); g.addColorStop(1, 'rgba(255,255,255,0)');
      x.fillStyle = g; x.fillRect(0, 0, 1280, 720);
    }
    x.restore();
    // glowing ring around the die
    if (L.ring > 0.01) {
      const ctr = P(0, yT, 0);
      x.save();
      const cg = x.createConicGradient(L.ringRot || 0, ctr[0], ctr[1]);
      ['#b69cff', '#e879f9', '#ff9a4d', '#5eead4', '#6d7cff', '#b69cff'].forEach((s, i, arr) =>
cg.addColorStop(i / (arr.length - 1), s));
      x.strokeStyle = cg; x.globalAlpha = Math.min(1, L.ring); x.lineWidth = 2.2 * sc;
      x.shadowColor = 'rgba(200,130,255,0.9)'; x.shadowBlur = 16 * sc * Math.min(1.6, L.ring);
      poly(x, die); x.stroke();
      x.shadowBlur = 0; x.lineWidth = 0.8 * sc; x.strokeStyle = `rgba(255,255,255,${0.5 *
Math.min(1, L.ring)})`; poly(x, die); x.stroke();
      x.restore();
    }
    // top chamfer edge: rim-lit at the back, key-lit at the front
    x.lineWidth = 1.5 * sc;
    for (const s of segs) {
      const back = Math.max(0, -s.NZ), front = Math.max(0, s.NX * -0.6 + s.NZ * 0.8);
      const a = Math.min(1, Math.pow(back, 1.2) * 0.95 * L.rim + front * 0.45 * L.key + 0.06);
      x.strokeStyle = `rgba(235,238,255,${a})`;
      x.beginPath(); x.moveTo(top[s.i][0], top[s.i][1]); x.lineTo(top[s.j][0], top[s.j][1]); x.stroke();
    }
  }
  let RL = null;
  function chip(x, c, L) {
    const u = c.unit, white = !!L.white;
    // contact shadow + coloured light spill on the floor
    x.save(); x.translate(c.cx, c.cy); x.scale(1, Math.max(0.18, Math.sin(c.pitch) * 0.8));
    let g = x.createRadialGradient(0, 0, 0, 0, 0, u * 1.45);
    g.addColorStop(0, `rgba(0,0,0,${white ? 0.32 : 0.75})`); g.addColorStop(0.6,
`rgba(0,0,0,${white ? 0.1 : 0.3})`); g.addColorStop(1, 'rgba(0,0,0,0)');
    x.fillStyle = g; x.beginPath(); x.arc(0, 0, u * 1.45, 0, 7); x.fill();
    if (L.ring > 0.01 && !white) {
      g = x.createRadialGradient(0, 0, 0, 0, 0, u * 1.9);
      g.addColorStop(0, `rgba(170,110,255,${0.16 * Math.min(1.5, L.ring)})`);
g.addColorStop(0.5, `rgba(255,130,90,${0.05 * Math.min(1.5, L.ring)})`); g.addColorStop(1,
'rgba(0,0,0,0)');
      x.globalCompositeOperation = 'lighter'; x.fillStyle = g; x.beginPath(); x.arc(0, 0, u * 1.9, 0,
7); x.fill();
    }
    x.restore();
    // floor reflection
    if (!RL) RL = H.layer(1280, 720);
```

```
    const r = RL.getContext('2d'); r.setTransform(1, 0, 0, 1, 0, 0); r.globalCompositeOperation
= 'source-over'; r.clearRect(0, 0, 1280, 720);
    body(r, c, L, true);
    r.globalCompositeOperation = 'destination-in';
    g = r.createLinearGradient(0, c.cy, 0, c.cy + u * 0.75);
    g.addColorStop(0, 'rgba(0,0,0,1)'); g.addColorStop(1, 'rgba(0,0,0,0)');
    r.fillStyle = g; r.fillRect(0, 0, 1280, 720);
    x.save(); x.globalAlpha = white ? 0.22 : 0.45 * Math.max(0.2, L.key); x.drawImage(RL, 0,
0, 1280, 720); x.restore();
    body(x, c, L, false);
  }

  // ---------- type ----------
  function setF(x, w, px) { x.font = `${w} ${px}px "Inter"`; try { x.letterSpacing = (px >= 40 ?
-Math.round(px * 0.035) : 0) + 'px'; } catch (e) {} }
  function grad(x, x0, x1, stops) { const g = x.createLinearGradient(x0, 0, x1, 0);
stops.forEach((s, i) => g.addColorStop(i / (stops.length - 1), s)); return g; }
  function wordsW(x, list, w, px) { setF(x, w, px); const sp = x.measureText(' ').width + px *
0.08; let tot = 0; list.forEach((it, i) => { it._w = x.measureText(it.s).width; tot += it._w + (i ? sp :
0); }); return { tot, sp }; }
  // one-word-at-a-time reveal: focus-pull from blur, rise and fade. list: [{s, t, fill?}]
  function words(x, list, X, Y, w, px, t, o) {
    o = o || {};
    x.save();
    const { tot, sp } = wordsW(x, list, w, px);
    let cx = o.align === 'center' ? X - tot / 2 : o.align === 'right' ? X - tot : X;
    const x0 = cx;
    for (const it of list) {
      const e = eo((t - it.t) / (o.dur || 0.6));
      if (e > 0.001) {
        x.globalAlpha = e * (o.alpha == null ? 1 : o.alpha);
        const b = (1 - e) * (o.blur == null ? 10 : o.blur);
        x.filter = b > 0.3 ? `blur(${b.toFixed(1)}px)` : 'none';
        x.fillStyle = it.fill === 'grad' ? grad(x, o.gx0 == null ? x0 : o.gx0, o.gx1 == null ? x0 + tot :
o.gx1, o.stops || GV) : it.fill || o.fill || INK;
        x.textAlign = 'left'; x.textBaseline = 'alphabetic';
        x.fillText(it.s, cx, Y + (1 - e) * px * 0.28);
      }
      cx += it._w + sp;
    }
    x.restore();
    return { x0, tot };
  }
  function label(x, s, X, Y, w, px, fill, align, a) {
    x.save(); setF(x, w, px); x.globalAlpha = a == null ? 1 : a; x.fillStyle = fill; x.textAlign = align
|| 'left'; x.textBaseline = 'alphabetic'; x.fillText(s, X, Y); x.restore();
  }
  // hairline callout: anchor dot -> elbow -> end, label sits at the end
```

```
  function callout(x, pts, p, lab, sub, side, o) {
    o = o || {};
    if (p <= 0) return;
    const ink = o.ink || '255,255,255';
    x.save();
    const [A] = pts, e = eo(p / 0.35);
    x.fillStyle = `rgba(${ink},${0.95 * e})`; x.beginPath(); x.arc(A[0], A[1], 3.6, 0, 7); x.fill();
    x.strokeStyle = `rgba(${ink},${0.35 * e})`; x.lineWidth = 1; x.beginPath(); x.arc(A[0], A[1], 5
+ 6 * e, 0, 7); x.stroke();
    let len = 0; for (let i = 1; i < pts.length; i++) len += Math.hypot(pts[i][0] - pts[i - 1][0], pts[i][1]
- pts[i - 1][1]);
    let rem = len * io(sm(0.1, 0.75, p));
    x.strokeStyle = `rgba(${ink},0.7)`; x.lineWidth = 1; x.beginPath(); x.moveTo(A[0], A[1]);
    for (let i = 1; i < pts.length && rem > 0; i++) {
      const a = pts[i - 1], b = pts[i], d = Math.hypot(b[0] - a[0], b[1] - a[1]), f = Math.min(1, rem /
d);
      x.lineTo(a[0] + (b[0] - a[0]) * f, a[1] + (b[1] - a[1]) * f); rem -= d;
    }
    x.stroke();
    const la = sm(0.6, 1, p), E = pts[pts.length - 1], dx = (1 - la) * 10 * (side === 'r' ? 1 : -1);
    if (la > 0) {
      const tx = side === 'r' ? E[0] + 14 + dx : E[0] - 14 + dx, al = side === 'r' ? 'left' : 'right';
      label(x, lab, tx, E[1] + 10, 600, o.px || 28, o.labFill || `rgb(${ink})`, al, la);
      if (sub) label(x, sub, tx, E[1] + 10 + (o.px || 28) * 1.05, 400, o.subPx || 21, o.subFill ||
GREY, al, la);
    }
    x.restore();
  }
  // arrow along a polyline/bezier sample list, drawn to progress p
  function arrow(x, pts, p, stroke, lw, head) {
    if (p <= 0) return;
    const n = Math.max(2, Math.round(pts.length * H.clamp(p)));
    x.save(); x.strokeStyle = stroke; x.lineWidth = lw || 1.2; x.lineCap = 'round'; x.beginPath();
    for (let i = 0; i < n; i++) (i ? x.lineTo : x.moveTo).call(x, pts[i][0], pts[i][1]);
    x.stroke();
    if (head !== false && n >= 2) {
      const a = pts[n - 2], b = pts[n - 1], an = Math.atan2(b[1] - a[1], b[0] - a[0]), hl = 10;
      x.beginPath(); x.moveTo(b[0] - hl * Math.cos(an - 0.45), b[1] - hl * Math.sin(an - 0.45));
x.lineTo(b[0], b[1]); x.lineTo(b[0] - hl * Math.cos(an + 0.45), b[1] - hl * Math.sin(an + 0.45));
x.stroke();
    }
    x.restore();
  }
  const bez = (p0, p1, p2, p3, n) => Array.from({ length: n + 1 }, (_, i) => { const u = i / n, v = 1
- u; return [v * v * v * p0[0] + 3 * v * v * u * p1[0] + 3 * v * u * u * p2[0] + u * u * u * p3[0], v * v *
v * p0[1] + 3 * v * v * u * p1[1] + 3 * v * u * u * p2[1] + u * u * u * p3[1]]; });

  // ---------- example "photo" cards (cached sprites, pre-blurred for depth of field) ----------
```

```
  const CARD = {}, CW = 150, CH = 184, CS = 2, CP = 40;
  function cardSprite(kind, tag, tagCol, blur) {
    const key = [kind, tag, tagCol, blur].join('|');
    if (CARD[key]) return CARD[key];
    const c = H.layer(CW * CS + CP * 2, CH * CS + CP * 2), x = c.getContext('2d');
    if (blur) { const b = cardSprite(kind, tag, tagCol, 0); x.filter = `blur(${blur * CS}px)`;
x.drawImage(b, 0, 0); return (CARD[key] = c); }
    x.translate(CP, CP); x.scale(CS, CS);
    x.beginPath(); x.roundRect(0, 0, CW, CH, 20); x.save(); x.clip();
    let g = x.createLinearGradient(0, 0, CW * 0.4, CH);
    if (kind === 'cat') { g.addColorStop(0, '#ffd2a6'); g.addColorStop(1, '#ee7d55'); } else if
(kind === 'dog') { g.addColorStop(0, '#a9ccff'); g.addColorStop(1, '#5b6ff0'); } else {
g.addColorStop(0, '#ffd2a6'); g.addColorStop(1, '#ee7d55'); }
    x.fillStyle = g; x.fillRect(0, 0, CW, CH);
    g = x.createRadialGradient(CW * 0.5, 70, 10, CW * 0.5, 70, 110); g.addColorStop(0,
'rgba(255,255,255,0.35)'); g.addColorStop(1, 'rgba(255,255,255,0)'); x.fillStyle = g;
x.fillRect(0, 0, CW, CH);
    const bg = kind === 'dog' ? '#4d5fd6' : '#d8673f';
    x.fillStyle = '#fff';
    if (kind !== 'dog') {
      x.beginPath(); x.moveTo(44, 70); x.lineTo(40, 28); x.lineTo(68, 52); x.closePath(); x.fill();
      x.beginPath(); x.moveTo(106, 70); x.lineTo(110, 28); x.lineTo(82, 52); x.closePath();
x.fill();
      x.beginPath(); x.ellipse(75, 80, 40, 34, 0, 0, 7); x.fill();
      x.fillStyle = bg; x.beginPath(); x.ellipse(60, 77, 4.5, 7, 0, 0, 7); x.ellipse(90, 77, 4.5, 7, 0,
0, 7); x.fill();
      x.beginPath(); x.moveTo(70, 91); x.lineTo(80, 91); x.lineTo(75, 97); x.closePath(); x.fill();
      x.strokeStyle = bg; x.lineWidth = 1.6; x.lineCap = 'round'; x.beginPath();
      x.moveTo(75, 97); x.quadraticCurveTo(70, 103, 64, 100); x.moveTo(75, 97);
x.quadraticCurveTo(80, 103, 86, 100);
      x.moveTo(52, 92); x.lineTo(30, 88); x.moveTo(52, 97); x.lineTo(32, 100); x.moveTo(98,
92); x.lineTo(120, 88); x.moveTo(98, 97); x.lineTo(118, 100); x.stroke();
    } else {
      x.fillStyle = 'rgba(255,255,255,0.78)';
      x.beginPath(); x.ellipse(42, 76, 14, 30, 0.35, 0, 7); x.ellipse(108, 76, 14, 30, -0.35, 0, 7);
x.fill();
      x.fillStyle = '#fff'; x.beginPath(); x.ellipse(75, 76, 36, 38, 0, 0, 7); x.fill();
      x.beginPath(); x.ellipse(75, 100, 20, 15, 0, 0, 7); x.fill();
      x.fillStyle = bg; x.beginPath(); x.ellipse(61, 72, 4.5, 5.5, 0, 0, 7); x.ellipse(89, 72, 4.5, 5.5,
0, 0, 7); x.fill();
      x.beginPath(); x.ellipse(75, 92, 8, 5.5, 0, 0, 7); x.fill();
      x.strokeStyle = bg; x.lineWidth = 1.6; x.beginPath(); x.moveTo(75, 97); x.lineTo(75, 104);
x.quadraticCurveTo(69, 109, 64, 105); x.moveTo(75, 104); x.quadraticCurveTo(81, 109, 86,
105); x.stroke();
    }
    // tag pill
    x.font = '600 28px "Inter"';
    const tw = x.measureText(tag).width + 30;
```

```
    x.fillStyle = tagCol || 'rgba(10,10,14,0.55)'; x.beginPath(); x.roundRect(75 - tw / 2, 132, tw,
40, 20); x.fill();
    x.fillStyle = '#fff'; x.textAlign = 'center'; x.textBaseline = 'middle'; x.fillText(tag, 75, 153);
    // gloss
    g = x.createLinearGradient(0, 0, CW, CH * 0.7); g.addColorStop(0,
'rgba(255,255,255,0.28)'); g.addColorStop(0.45, 'rgba(255,255,255,0)'); x.fillStyle = g;
x.fillRect(0, 0, CW, CH);
    x.restore();
    x.strokeStyle = 'rgba(255,255,255,0.45)'; x.lineWidth = 1; x.beginPath(); x.roundRect(0.5,
0.5, CW - 1, CH - 1, 20); x.stroke();
    return (CARD[key] = c);
  }
  function drawCard(x, kind, tag, tagCol, cx, cy, s, blur, rot, a) {
    const lv = [0, 2, 5, 9]; let bi = 0; for (let i = 0; i < lv.length; i++) if (Math.abs(lv[i] - blur) <
Math.abs(lv[bi] - blur)) bi = i;
    const sp = cardSprite(kind, tag, tagCol, lv[bi]);
    x.save(); x.globalAlpha = a; x.translate(cx, cy); x.rotate(rot || 0); x.scale(s / CS, s / CS);
    x.drawImage(sp, -sp.width / 2, -sp.height / 2); x.restore();
  }

  // ---------- finishing ----------
  let GR = null;
  function finish(x, W, Hh, t, white) {
    if (!GR) { GR = H.layer(256, 256); const g = GR.getContext('2d'), img =
g.createImageData(256, 256), r = H.rng(77); for (let i = 0; i < img.data.length; i += 4) { const
v = r() * 255; img.data[i] = img.data[i + 1] = img.data[i + 2] = v; img.data[i + 3] = 255; }
g.putImageData(img, 0, 0); }
    if (!white) H.vignette(x, W, Hh, 0.55);
    const r = H.rng(900 + Math.floor(t * 24));
    x.save(); x.globalAlpha = white ? 0.04 : 0.06; x.globalCompositeOperation = 'soft-light';
    x.translate(-r() * 256, -r() * 256); x.fillStyle = x.createPattern(GR, 'repeat'); x.fillRect(0, 0, W
+ 256, Hh + 256); x.restore();
  }
  function ringGlow(base, t, pulses) { let v = base; for (const p of pulses) if (t > p) v += 0.9 *
Math.exp(-(t - p) * 5); return v; }

  // ================= STILL / CLIP =================
  function drawHero(ctx, W, Hh, t) {
    const D = 6, q = io(t / D);
    const c = cam({ cx: 640, cy: 572, s: H.lerp(0.68, 0.8, q), yaw: H.lerp(-0.95, -0.42, q), pitch:
H.lerp(0.32, 0.42, q) });
    const L = { key: sm(0.7, 2.2, t), rim: sm(0.1, 1.1, t), ring: sm(1.5, 2.1, t) + 0.8 *
Math.exp(-Math.max(0, t - 1.6) * 3) * (t > 1.6 ? 1 : 0), spec: H.map(t, 2.0, 3.8, -0.25, 1.25),
      fwd: t > 2.4 ? ((t - 2.4) % 1.8) / 1.4 : -1, back: -1, adj: 0, act: sm(1.6, 2.6, t), hover: 0.36 +
0.015 * Math.sin(t * 1.2), ringRot: t * 0.25 };
    if (L.fwd > 1) L.fwd = -1;
    stage(ctx, W, Hh, c, L, t);
    chip(ctx, c, L);
```

```
    // headline
    words(ctx, [{ s: 'Introducing', t: 1.75, fill: 'grad' }], 640, 92, 600, 26, t, { align: 'center', stops:
GV });
    words(ctx, [{ s: 'Thinks', t: 2.05 }, { s: 'in', t: 2.3 }, { s: 'layers.', t: 2.55 }], 640, 176, 800, 88,
t, { align: 'center' });
    // callouts on the object
    const yT = L.hover + THICK / 2;
    const a1 = c.P(NODES[0][1][0], yT, NODES[0][1][1]), a2 = c.P(DIE[3][0], yT, DIE[3][1]), a3
= c.P(0.55, L.hover - THICK * 0.2, 1.0);
    callout(ctx, [a1, [a1[0] - 60, 300], [380, 300]], sm(3.0, 3.9, t), '4-layer neural core', 'Learns
from every example', 'l');
    callout(ctx, [a2, [a2[0] + 50, 300], [920, 300]], sm(3.35, 4.25, t), 'Edge-lit glass', 'Sapphire
top, 0.4 mm', 'r');
    callout(ctx, [a3, [a3[0] + 40, 590], [920, 590]], sm(3.7, 4.6, t), 'Graphite unibody', 'Machined
from one block', 'r');
    finish(ctx, W, Hh, t, false);
  }

  // ================= EXPLAINER =================
  const D_EX = 15.5;
  // timing (shared by explainer() and explainerSound())
  const T = {
    hook: [0.3, 0.55, 0.8, 1.05], key: [1.15, 2.4], rim: [0.6, 1.5], ringOn: 1.95, head: [1.85, 2.6],
    path: [2.6, 3.1], cards: Array.from({ length: 14 }, (_, i) => 2.68 + i * 0.18), travel: 1.25, cnt:
[2.95, 5.5],
    inCard: 6.15, inArrow: [6.5, 6.8], fwd1: [6.8, 7.25], outArrow: [7.2, 7.45], dog: 7.45,
    check: 8.1, strike: [8.15, 8.4], xmark: 8.42, err: [8.5, 8.95], back: [8.92, 9.42], adjust: 8.95,
    fwd2: [9.45, 9.75], dogOut: [9.5, 9.72], cat: 9.75, loop: [9.95, 10.6],
    acc: 10.8, num: [11.0, 12.4], lock: 12.4, gleam: [12.45, 12.95],
    sweep: [13.0, 13.45], tag: [13.35, 13.52, 13.69, 13.86, 14.03],
  };
  const KEYS = [
    { t: 0, cx: 640, cy: 640, s: 0.58, yaw: -0.8, pitch: 0.30 },
    { t: 1.7, cx: 640, cy: 628, s: 0.62, yaw: -0.66, pitch: 0.32 },
    { t: 2.8, cx: 860, cy: 520, s: 0.8, yaw: -0.32, pitch: 0.42 },
    { t: 5.8, cx: 880, cy: 515, s: 0.85, yaw: -0.08, pitch: 0.46 },
    { t: 6.6, cx: 580, cy: 500, s: 0.76, yaw: 0.06, pitch: 0.72 },
    { t: 10.25, cx: 590, cy: 500, s: 0.79, yaw: 0.2, pitch: 0.74 },
    { t: 11.0, cx: 300, cy: 520, s: 0.64, yaw: -0.3, pitch: 0.42 },
    { t: 13.0, cx: 290, cy: 520, s: 0.68, yaw: -0.12, pitch: 0.42 },
    { t: 14.3, cx: 640, cy: 590, s: 0.7, yaw: 0.3, pitch: 0.36 },
    { t: 15.5, cx: 640, cy: 585, s: 0.72, yaw: 0.42, pitch: 0.36 },
  ];
  const accVal = (p) => 61 + 36 * (1 - Math.pow(1 - p, 2.2)) + Math.sin(p * 23) * 2.2 * (1 - p) *
p;

  function lighting(t) {
```

```
    const pulses = T.cards.map((c) => c + T.travel - 0.05).concat([T.fwd1[1], T.fwd2[1], T.lock,
T.sweep[1]]);
    let fwd = -1, back = -1;
    if (t > T.fwd1[0] && t < T.fwd1[1] + 0.2) fwd = (t - T.fwd1[0]) / (T.fwd1[1] - T.fwd1[0]);
    else if (t > T.fwd2[0] && t < T.fwd2[1] + 0.2) fwd = (t - T.fwd2[0]) / (T.fwd2[1] - T.fwd2[0]);
    else if (t > 2.8 && t < 6.4) fwd = ((t - 2.8) % 0.72) / 0.6;
    else if (t > 11.0 && t < 12.4) fwd = ((t - 11.0) % 0.47) / 0.4;
    if (fwd > 1.15) fwd = -1;
    if (t > T.back[0] && t < T.back[1] + 0.2) back = (t - T.back[0]) / (T.back[1] - T.back[0]);
    return { key: sm(T.key[0], T.key[1], t), rim: sm(T.rim[0], T.rim[1], t), ring: t < T.ringOn ? 0 :
ringGlow(sm(T.ringOn, T.ringOn + 0.5, t) * 0.85, t, pulses),
      spec: t < 8 ? H.map(t, 2.2, 4.0, -0.25, 1.25) : t < 13.3 ? H.map(t, 12.35, 13.0, -0.25, 1.25)
: H.map(t, 13.6, 14.9, -0.25, 1.25),
      fwd, back, adj: io(sm(T.back[0], T.back[1] + 0.1, t)), act: sm(1.9, 2.8, t), hover: 0.36 +
0.015 * Math.sin(t * 1.2), ringRot: t * 0.3 };
  }

  function darkScene(x, W, Hh, t, c, L) {
    stage(x, W, Hh, c, L, t);
    const yT = L.hover + THICK / 2, top = c.P(0, yT, 0);
    // loop ring (back half before the chip)
    const ringP = sm(T.loop[0], T.loop[1], t), ringA = (1 - sm(T.loop[1], T.loop[1] + 0.35, t));
    if (ringP > 0 && ringA > 0) loopRing(x, c, L, ringP, ringA, false);
    chip(x, c, L);
    if (ringP > 0 && ringA > 0) loopRing(x, c, L, ringP, ringA, true);

    // ---- beat 1: hook, then shrink to a corner header
    {
      const list = [{ s: 'How', t: T.hook[0] }, { s: 'does', t: T.hook[1] }, { s: 'AI', t: T.hook[2] }, { s:
'learn?', t: T.hook[3] }];
      const m = io(sm(T.head[0], T.head[1], t)), px = 108;
      const { tot } = wordsW(x, list, 800, px);
      const k = H.lerp(1, 44 / px, m), X = H.lerp(640 - tot / 2, 56, m), Y = H.lerp(330, 82, m);
      const fade = 1 - sm(12.6, 13.0, t);
      if (fade > 0) {
        x.save(); x.translate(X, Y); x.scale(k, k);
        words(x, list, 0, 0, 800, px, t, { alpha: H.lerp(1, 0.9, m) * fade });
        x.restore();
      }
    }

    // ---- beat 2: training data streams into the model
    const b2 = 1 - sm(5.85, 6.35, t), dx2 = c.cx - 880;
    if (t > T.path[0] && b2 > 0) {
      x.save(); x.globalAlpha = b2;
      const P0 = [-160, 690 + 0], P1 = [80, 380], P2 = [360, 230], P3 = [top[0] - 30, top[1] - 6];
      const path = bez(P0, P1, P2, P3, 60).map((p) => [p[0] + (p === null ? 0 : 0), p[1]]);
      arrow(x, path.slice(8, 57), sm(T.path[0], T.path[1], t), 'rgba(255,255,255,0.32)', 1.2);
```

```
    callout(x, [[path[24][0], path[24][1]], [path[24][0] + 30, 172], [250, 172]], sm(2.95, 3.8, t),
'Training data', 'labeled examples', 'r');
    const kinds = ['cat', 'dog', 'dog', 'cat', 'cat', 'dog', 'cat', 'dog', 'dog', 'cat', 'dog', 'cat', 'cat',
'dog'];
    // draw far-to-near: later cards (closer to camera at start) on top
    for (let i = 0; i < T.cards.length; i++) {
      const u = (t - T.cards[i]) / T.travel;
      if (u <= 0 || u >= 1) continue;
      const v = H.ease.inOut(u) * 0.9 + u * 0.1;
      const pts = bez(P0, P1, P2, P3, 1), p = (() => { const w = 1 - v; return [w * w * w * P0[0]
+ 3 * w * w * v * P1[0] + 3 * w * v * v * P2[0] + v * v * v * P3[0], w * w * w * P0[1] + 3 * w * w *
v * P1[1] + 3 * w * v * v * P2[1] + v * v * v * P3[1]]; })();
      void pts;
      const s = v < 0.42 ? H.lerp(1.7, 1.05, v / 0.42) : H.lerp(1.05, 0.08, Math.pow((v - 0.42) /
0.58, 1.25));
      const blur = v < 0.38 ? H.lerp(9, 0, v / 0.38) : v > 0.85 ? (v - 0.85) * 20 : 0;
      const a = Math.min(1, u * 6) * (1 - sm(0.86, 1, v));
      drawCard(x, kinds[i], kinds[i], null, p[0], p[1], s, blur, H.lerp(-0.22, 0.12, v) + Math.sin(i *
2.1) * 0.06, a);
    }
    // counter
    const cv = Math.round(Math.pow(10, 6 * io(sm(T.cnt[0], T.cnt[1], t))));
    const ca = sm(T.cnt[0] - 0.2, T.cnt[0] + 0.15, t);
    if (ca > 0) {
      x.save(); x.globalAlpha = ca * b2; setF(x, 800, 80);
      const s = cv.toLocaleString('en-US'), w = x.measureText(s).width, X = 1216 + dx2 * 0.3 -
w;
      x.shadowColor = 'rgba(60,170,255,0.55)'; x.shadowBlur = 30; x.fillStyle = grad(x, X, X +
w, GT); x.fillText(s, X, 636);
      x.shadowBlur = 0; x.fillText(s, X, 636);
      label(x, 'examples', 1216 + dx2 * 0.3, 680, 400, 30, GREY, 'right');
      x.restore();
    }
    x.restore();
  }

  // ---- beat 3: guess -> check -> adjust
  const b3 = sm(6.05, 6.5, t) * (1 - sm(10.3, 10.65, t)), dx3 = c.cx - 580;
  if (b3 > 0) {
    x.save(); x.globalAlpha = b3; x.translate(dx3 * 0.9, 0);
    const ix = 172, iy = 396;
    const cardIn = eo(sm(T.inCard, T.inCard + 0.55, t));
    const flip = sm(T.check, T.check + 0.25, t);
    const cs = H.lerp(0.9, 1.18, cardIn), cb = (1 - cardIn) * 8;
    drawCard(x, 'cat', '?', null, ix - (1 - cardIn) * 80, iy, cs, cb, -0.04, cardIn * (1 - flip));
    if (flip > 0) drawCard(x, 'cat', 'cat', 'rgba(32,190,140,0.95)', ix, iy, cs, 0, -0.04, flip);
    callout(x, [[ix, iy - 112], [ix, 232], [ix + 24, 232]], sm(T.check, T.check + 0.7, t), '02  Check',
'it\'s a cat', 'r');
```

```
    const L0 = c.P(-1.0, yT, 0.05), R0 = c.P(1.0, yT, -0.05);
    arrow(x, bez([ix + 96, iy], [ix + 150, iy], [L0[0] - 70, L0[1]], [L0[0] - 14 - dx3 * 0.9, L0[1]],
24), sm(T.inArrow[0], T.inArrow[1], t), 'rgba(255,255,255,0.6)', 1.3);
    const ox = 930, oy = 404;
    arrow(x, bez([R0[0] + 14 - dx3 * 0.9, R0[1]], [R0[0] + 40 - dx3 * 0.9, R0[1]], [ox - 60, oy -
34], [ox - 18, oy - 34], 24), sm(T.outArrow[0], T.outArrow[1], t), 'rgba(255,255,255,0.6)', 1.3);
    // guess
    const out1 = 1 - sm(T.dogOut[0], T.dogOut[1], t);
    if (t > T.dog && out1 > 0) {
      x.save(); x.globalAlpha *= out1; x.translate(0, -(1 - out1) * 30);
      const r = words(x, [{ s: 'dog?', t: T.dog }], ox, oy, 800, 104, t, {});
      const sp = sm(T.strike[0], T.strike[1], t);
      if (sp > 0) {
        x.strokeStyle = RED; x.lineWidth = 7; x.lineCap = 'round'; x.shadowColor =
'rgba(255,69,58,0.6)'; x.shadowBlur = 14;
        x.beginPath(); x.moveTo(ox - 8, oy - 34); x.lineTo(ox - 8 + (r.tot + 16) * io(sp), oy - 34);
x.stroke(); x.shadowBlur = 0;
      }
      const xm = eo(sm(T.xmark, T.xmark + 0.3, t));
      if (xm > 0) {
        const mx = ox + r.tot + 52, my = oy - 36, rr_ = 26 * (0.6 + 0.4 * xm);
        x.globalAlpha *= xm; x.strokeStyle = RED; x.lineWidth = 3.5;
        x.beginPath(); x.arc(mx, my, rr_, 0, 7); x.stroke();
        x.beginPath(); x.moveTo(mx - 9, my - 9); x.lineTo(mx + 9, my + 9); x.moveTo(mx + 9,
my - 9); x.lineTo(mx - 9, my + 9); x.stroke();
      }
      x.restore();
    }
    if (t > T.cat) {
      const r = words(x, [{ s: 'cat', t: T.cat, fill: 'grad' }], ox, oy, 800, 104, t, { stops: GG });
      x.save(); x.shadowColor = 'rgba(60,230,170,0.6)'; x.shadowBlur = 24 * Math.exp(-(t -
T.cat) * 2);
      const cm = eo(sm(T.cat + 0.1, T.cat + 0.4, t)), mx = ox + r.tot + 52, my = oy - 36;
      x.globalAlpha *= cm; x.fillStyle = '#2fd6a0'; x.beginPath(); x.arc(mx, my, 26 * (0.6 + 0.4 *
cm), 0, 7); x.fill();
      x.shadowBlur = 0; x.strokeStyle = '#04120c'; x.lineWidth = 4; x.lineCap = 'round';
x.lineJoin = 'round';
      x.beginPath(); x.moveTo(mx - 11, my + 1); x.lineTo(mx - 3, my + 9); x.lineTo(mx + 12,
my - 8); x.stroke();
      x.restore();
    }
    callout(x, [[ox + 4, oy - 108], [ox + 4, 238], [ox + 28, 238]], sm(T.dog + 0.05, T.dog + 0.75,
t), '01  Guess', null, 'r');
    // error signal travelling back to the model
    const ep = sm(T.err[0], T.err[1], t), ea = 1 - sm(T.back[1], T.back[1] + 0.4, t);
    if (ep > 0 && ea > 0) {
      const F0 = c.P(0.55, yT - THICK, 1.0);
```

```
      const epath = bez([ox + 70, oy + 28], [ox + 60, 640], [F0[0] + 180 - dx3 * 0.9, 650],
[F0[0] + 10 - dx3 * 0.9, F0[1] + 26], 50);
      x.save(); x.globalAlpha *= ea;
      arrow(x, epath, ep, 'rgba(255,110,80,0.75)', 1.6);
      // the struck word charges with a soft red underglow, and the signal leaves it as a small
comet with a tail
      { const ug = (1 - sm(T.err[0] + 0.25, T.back[1], t)) * 0.2;
        if (ug > 0.005) { x.save(); x.translate(ox + 150, oy - 30); x.scale(1, 0.32); const g =
x.createRadialGradient(0, 0, 0, 0, 0, 240); g.addColorStop(0, `rgba(255,70,50,${ug})`);
g.addColorStop(1, 'rgba(255,60,40,0)'); x.globalCompositeOperation = 'lighter'; x.fillStyle = g;
x.beginPath(); x.arc(0, 0, 240, 0, 7); x.fill(); x.restore(); } }
      if (ep < 1) {
        const hi = Math.min(epath.length - 1, Math.round(ep * (epath.length - 1))), hp =
epath[hi], ha = sm(0, 0.14, ep);
        const ti = Math.max(0, hi - 7), tp = epath[ti];
        if (hi > ti) { const tg = x.createLinearGradient(tp[0], tp[1], hp[0], hp[1]);
tg.addColorStop(0, 'rgba(255,90,60,0)'); tg.addColorStop(1, `rgba(255,150,120,${0.9 * ha})`);
x.strokeStyle = tg; x.lineWidth = 3; x.lineCap = 'round'; x.beginPath(); for (let q = ti; q <= hi;
q++) (q > ti ? x.lineTo : x.moveTo).call(x, epath[q][0], epath[q][1]); x.stroke(); }
        const g = x.createRadialGradient(hp[0], hp[1], 0, hp[0], hp[1], 18); g.addColorStop(0,
`rgba(255,225,205,${ha})`); g.addColorStop(0.3, `rgba(255,100,70,${0.75 * ha})`);
g.addColorStop(1, 'rgba(255,60,40,0)'); x.fillStyle = g; x.beginPath(); x.arc(hp[0], hp[1], 18, 0,
7); x.fill();
      }
      label(x, 'error', epath[22][0] + 18, epath[22][1] + 8, 600, 28, '#ff7a5c', 'left', sm(T.err[0] +
0.15, T.err[0] + 0.4, t));
      x.restore();
    }
    // adjust callout on the inner layers
    const an = c.P(NODES[2][1][0], yT, NODES[2][1][1]);
    callout(x, [[an[0] - dx3 * 0.9, an[1]], [an[0] + 30 - dx3 * 0.9, 150], [an[0] + 70 - dx3 * 0.9,
150]], sm(T.adjust, T.adjust + 0.75, t), '03  Adjust', 'weights shift', 'r', { ink: '255,190,150' });
    if (ringP > 0.05 && ringA > 0) { const fr = c.P(1.42 * Math.cos(0.75), L.hover, 1.42 *
Math.sin(0.75)); label(x, 'Repeat × 1,000,000', fr[0] + 22 - dx3 * 0.9, fr[1] + 10, 600, 30, INK,
'left', sm(T.loop[0] + 0.05, T.loop[0] + 0.3, t) * ringA); }
    x.restore();
  }

  // ---- beat 4: accuracy, the giant gradient number
  const b4 = sm(10.6, 11.0, t);
  if (b4 > 0) {
    const p = eo(sm(T.num[0], T.num[1], t)), val = Math.round(accVal(p));
    const lk = t > T.lock ? Math.exp(-(t - T.lock) * 2.2) : 0;
    const zoom = 1 + 0.05 * sm(T.lock, T.lock + 0.5, t) + 0.035 * lk;
    x.save(); x.globalAlpha = b4;
    label(x, 'Accuracy', 566, 196, 600, 36, GREY, 'left', sm(T.acc, T.acc + 0.4, t));
    // chart
    const cx0 = 572, cx1 = 1196, yOf = (v) => 640 - ((v - 55) / 45) * 170, N = 60;
```

```
    x.strokeStyle = 'rgba(255,255,255,0.18)'; x.lineWidth = 1; x.beginPath(); x.moveTo(cx0,
642); x.lineTo(cx0 + (cx1 - cx0) * sm(10.85, 11.3, t), 642); x.stroke();
    const cp = sm(T.num[0], T.num[1], t);
    const pts = []; for (let i = 0; i <= N * cp; i++) { const r = i / N; pts.push([H.lerp(cx0, cx1, r),
yOf(accVal(r))]); }
    if (pts.length > 1) {
      const hp = [H.lerp(cx0, cx1, cp), yOf(accVal(cp))]; pts.push(hp);
      x.beginPath(); pts.forEach((q, i) => (i ? x.lineTo(q[0], q[1]) : x.moveTo(q[0], q[1])));
x.lineTo(hp[0], 642); x.lineTo(cx0, 642); x.closePath();
      let g = x.createLinearGradient(0, 440, 0, 642); g.addColorStop(0,
'rgba(232,121,249,0.22)'); g.addColorStop(1, 'rgba(232,121,249,0)'); x.fillStyle = g; x.fill();
      x.beginPath(); pts.forEach((q, i) => (i ? x.lineTo(q[0], q[1]) : x.moveTo(q[0], q[1])));
      x.strokeStyle = grad(x, cx0, cx1, GV); x.lineWidth = 3; x.lineJoin = 'round';
x.shadowColor = 'rgba(232,121,249,0.7)'; x.shadowBlur = 14; x.stroke(); x.shadowBlur = 0;
      g = x.createRadialGradient(hp[0], hp[1], 0, hp[0], hp[1], 22); g.addColorStop(0,
'rgba(255,255,255,1)'); g.addColorStop(0.3, 'rgba(255,170,110,0.8)'); g.addColorStop(1,
'rgba(255,150,80,0)'); x.fillStyle = g; x.beginPath(); x.arc(hp[0], hp[1], 22, 0, 7); x.fill();
    }
    label(x, 'Training rounds →', cx0, 682, 400, 28, '#8e8e93', 'left', sm(10.9, 11.3, t));
    // number
    const s = val + '%', na = sm(T.num[0] - 0.25, T.num[0] + 0.1, t);
    x.save(); x.translate(566, 404); x.scale(zoom, zoom); x.globalAlpha = b4 * na; setF(x,
800, 200);
    const w = x.measureText(s).width;
    const fill = grad(x, 0, w, GV);
    x.shadowColor = `rgba(220,110,255,${0.5 + 0.4 * lk})`; x.shadowBlur = 50 + 60 * lk;
x.fillStyle = fill; x.fillText(s, 0, 0);
    x.shadowBlur = 0; x.fillText(s, 0, 0);
    x.restore();
    // gleam across the number
    const gp = sm(T.gleam[0], T.gleam[1], t);
    if (gp > 0 && gp < 1) {
      const Lr = gleamLayer(); const g2 = Lr.getContext('2d'); g2.setTransform(1, 0, 0, 1, 0,
0); g2.globalCompositeOperation = 'source-over'; g2.clearRect(0, 0, 1280, 720);
      g2.save(); g2.translate(566, 404); g2.scale(zoom, zoom); setF(g2, 800, 200); g2.fillStyle
= '#fff'; g2.fillText(s, 0, 0); g2.restore();
      g2.globalCompositeOperation = 'source-in';
      const bx = 566 - 200 + gp * (w * zoom + 400), gg = g2.createLinearGradient(bx - 90, 0,
bx + 90, 0);
      gg.addColorStop(0, 'rgba(255,255,255,0)'); gg.addColorStop(0.5,
'rgba(255,255,255,0.85)'); gg.addColorStop(1, 'rgba(255,255,255,0)');
      g2.fillStyle = gg; g2.fillRect(0, 0, 1280, 720);
      x.drawImage(Lr, 0, 0, 1280, 720);
    }
    x.restore();
  }
 }
 let GL = null; const gleamLayer = () => GL || (GL = H.layer(1280, 720));
```

```
  function loopRing(x, c, L, p, a, front) {
    const N = 90, R = 1.42, y = L.hover, a0 = 1.6, cd = c.P(0, L.hover, 0)[2];
    const pts = []; for (let i = 0; i <= N * p; i++) { const an = a0 + (i / N) * PI * 2; pts.push(c.P(R
* Math.cos(an), y, R * Math.sin(an))); }
    if (pts.length < 2) return;
    x.save(); x.globalAlpha *= a; x.lineCap = 'round'; x.lineWidth = 2;
    const g = grad(x, c.cx - c.unit * 1.8, c.cx + c.unit * 1.8, GV); x.strokeStyle = g;
x.shadowColor = 'rgba(220,120,255,0.6)'; x.shadowBlur = 10;
    for (let i = 1; i < pts.length; i++) { if ((pts[i][2] > cd) !== front) continue; x.beginPath();
x.moveTo(pts[i - 1][0], pts[i - 1][1]); x.lineTo(pts[i][0], pts[i][1]); x.stroke(); }
    const hp = pts[pts.length - 1];
    if ((hp[2] > cd) === front) { const b = pts[pts.length - 2], an = Math.atan2(hp[1] - b[1], hp[0]
- b[0]); x.fillStyle = '#ffd0f0'; x.beginPath(); x.moveTo(hp[0] + 12 * Math.cos(an), hp[1] + 12 *
Math.sin(an)); x.lineTo(hp[0] + 9 * Math.cos(an + 2.3), hp[1] + 9 * Math.sin(an + 2.3));
x.lineTo(hp[0] + 9 * Math.cos(an - 2.3), hp[1] + 9 * Math.sin(an - 2.3)); x.fill(); }
    x.restore();
  }

  function whiteScene(x, W, Hh, t, c, L) {
    const Lw = Object.assign({}, L, { white: 1, key: 1, rim: 1 });
    stage(x, W, Hh, c, Lw, t);
    chip(x, c, Lw);
    const list = [{ s: 'More', t: T.tag[0], fill: '#1d1d1f' }, { s: 'examples', t: T.tag[1], fill: '#1d1d1f' },
{ s: '→', t: T.tag[2], fill: '#86868b' }, { s: 'better', t: T.tag[3], fill: 'grad' }, { s: 'guesses', t: T.tag[4],
fill: 'grad' }];
    const { tot, sp } = wordsW(x, list, 800, 66);
    const gx0 = 640 - tot / 2 + list[0]._w + list[1]._w + list[2]._w + sp * 3;
    words(x, list, 640, 196, 800, 66, t, { align: 'center', gx0, gx1: 640 + tot / 2, stops: ['#7c4dff',
'#d946ef', '#ff7a2f'] });
    label(x, 'That’s how AI learns.', 640, 262, 400, 30, '#6e6e73', 'center', sm(14.45, 14.9, t));
  }

  function explainer(ctx, W, Hh, t) {
    const c = cam(keyed(KEYS, t));
    const L = lighting(t);
    const s0 = T.sweep[0], s1 = T.sweep[1];
    if (t < s1) darkScene(ctx, W, Hh, t, c, L);
    if (t >= s0) {
      const bx = H.lerp(-260, W + 260, io(sm(s0, s1, t)));
      ctx.save();
      if (t < s1) { ctx.beginPath(); ctx.rect(0, 0, Math.max(0, bx), Hh); ctx.clip(); }
      whiteScene(ctx, W, Hh, t, c, L);
      ctx.restore();
      if (t < s1 + 0.05) {
        const g = ctx.createLinearGradient(bx - 260, 0, bx + 120, 0);
```

```
      g.addColorStop(0, 'rgba(255,255,255,0)'); g.addColorStop(0.7,
'rgba(255,255,255,0.9)'); g.addColorStop(0.85, 'rgba(255,255,255,0.55)'); g.addColorStop(1,
'rgba(255,255,255,0)');
      ctx.save(); ctx.globalCompositeOperation = 'lighter'; ctx.fillStyle = g; ctx.fillRect(bx -
260, 0, 380, Hh); ctx.restore();
    }
   }
   finish(ctx, W, Hh, t, t >= s1);
 }

 registerStyle({
   id: 'product-launch',
   name: 'Product Launch Reveal',
   category: 'Popular Explainer Looks',
   description: 'A big-tech keynote product film: one glossy hero object on a near-black
stage, lit like a studio shot with soft key, rim light, a sweeping specular highlight and a floor
reflection, huge tight-tracked sans headlines, gradient key numbers with a soft glow and
hairline spec callouts.',
   motion: 'Rim light finds the object in the dark, the key light fades up as the camera slowly
orbits and pushes in, a highlight glides across the glass, the edge ring ignites, then the
headline arrives one word at a time and hairline callouts draw out to their spec labels.',
   fonts: ['800 108px "Inter"', '800 88px "Inter"', '800 200px "Inter"', '600 28px "Inter"', '600
36px "Inter"', '400 30px "Inter"', '400 21px "Inter"'],
   frameT: 5.6,
   duration: 6,
   draw: drawHero,
   explainerDuration: D_EX,
   explainerNote: 'Told as a keynote launch film around one hero object (the model, a
glass-topped chip): one continuous studio camera that orbits and pushes between beats,
hairline callouts and arrows for flow, a clean red strike for the wrong guess while the chip’s
inner layers light up backwards to adjust, a giant violet-to-orange 97% with a glow bloom and
gleam, and a light-sweep wipe to a pristine white stage for the tagline.',
   explainer,
   // Keynote-film score: a quiet pad, then a minimal synth pulse under the data/learning
beats, deep soft sub "bwooms"
   // on reveals, airy risers into the big moments, clean low clicks for labels and words,
smooth low whooshes for every
   // camera move, a muted low two-note for the wrong guess, warm chords for "cat" and the
97% lock, and a resolving
   // chord under the tagline. All times come from T / the same easing used in explainer().
   explainerSound(k) {
    const D = D_EX, when = (f, v, a, b) => { for (let t = a; t <= b; t += 0.001) if (f(t) >= v) return
t; return b; };
     const bwoom = (t, g, dur = 1.4, f = 58) => {
      k.tone(t, { f: [[0, f * 1.7], [0.1, f], [dur, f * 0.72]], dur, gain: 0.5 * g, a: 0.012, hold: dur *
0.15 });
      k.tone(t, { f: f * 2, dur: dur * 0.45, gain: 0.07 * g, a: 0.012 });
```

```
    k.noise(t, { dur: 0.6, color: 'brown', filter: 'lowpass', f: [[0, 420], [0.6, 80]], gain: 0.22 * g, a: 0.008, verb: 0.25 });
  };
  const air = (t, dur, g) => { k.noise(t, { dur, color: 'pink', filter: 'bandpass', f: [[0, 450], [dur, 4200]], q: 0.7, gain: 0.13 * g, a: dur * 0.92, verb: 0.35 }); k.tone(t, { f: [[0, 'D3'], [dur, 'D4']], type: 'triangle', dur, gain: 0.025 * g, a: dur * 0.9, filter: 'lowpass', f2: 900 }); };
  const click = (t, g = 1, pan = 0, minor = true) => { k.noise(t, { dur: 0.014, filter: 'bandpass', f: 2300, q: 1.6, gain: 0.2 * g, pan, minor }); k.tone(t, { f: [[0, 420], [0.03, 260]], dur: 0.04, gain: 0.07 * g, pan, minor }); };
  const glide = (t, dur, g, pan = 0, minor = true) => k.noise(t, { dur, color: 'pink', filter: 'lowpass', f: [[0, 160], [dur * 0.5, 720], [dur, 190]], q: 0.9, gain: 0.26 * g, a: dur * 0.45, pan, verb: 0.15, minor });
  const chord = (t, notes, dur, g, minor = false) => notes.forEach((n, i) => {
    const pan = (i - (notes.length - 1) / 2) * 0.22;
    k.tone(t, { f: n, type: 'triangle', dur, gain: 0.05 * g, a: 0.03, hold: dur * 0.3, filter: 'lowpass', f2: [[0, 2200], [dur, 700]], pan, verb: 0.4, minor });
    k.tone(t, { f: k.hz(n) * 1.003, dur, gain: 0.04 * g, a: 0.05, hold: dur * 0.3, pan: -pan, verb: 0.4, minor });
  });
  const ticks = (val, a, b, gap, g, pan) => { let last = -1, prev = val(a); for (let t = a; t <= b; t += 0.002) { const v = val(t); if (v !== prev && t - last >= gap) { click(t, g, pan); last = t; } prev = v; } };

  // ---- music: quiet pad, minimal pulse through the middle, pad again for the resolve
  const CH = [['D3', 'A3', 'E4', 'F#4'], ['B2', 'F#3', 'D4', 'A4'], ['G2', 'D3', 'B3', 'F#4'], ['A2', 'E3', 'C#4', 'E4']];
  k.music('pad', 0.2, 2.65, { bpm: 64, gain: 0.32, chords: [CH[0]] });
  k.music('pulse', 2.65, 12.4, { bpm: 64, gain: 0.26, chords: [CH[1], CH[2], CH[3]] });
  k.music('pad', 12.4, D, { bpm: 64, gain: 0.3, chords: [CH[0]] });

  // ---- beat 1: words, light finds the object
  T.hook.forEach((t, i) => click(t, 0.8, (i - 1.5) * 0.15));
  air(0.4, 0.8, 0.8);
  bwoom(T.key[0] + 0.05, 0.7, 1.6);
  chord(T.ringOn, ['A3', 'E4'], 1.0, 0.5, true);                    // edge ring ignites
  glide(1.75, 1.05, 1.0, 0, false);                                 // header slides up, camera orbits right

  // ---- beat 2: data into the model
  click(T.path[1], 0.7, -0.3);
  click(when((t) => sm(2.95, 3.8, t), 0.6, 2.95, 3.8), 0.8, -0.4);       // "Training data" label lands
  T.cards.forEach((c) => { const ta = c + T.travel - 0.05; k.tone(ta, { f: [[0, 200], [0.12, 105]], dur: 0.16, gain: 0.12, pan: 0.3, minor: true }); k.noise(ta, { dur: 0.06, color: 'brown', filter: 'lowpass', f: 600, gain: 0.12, pan: 0.3, minor: true }); });
  ticks((t) => Math.round(Math.pow(10, 6 * io(sm(T.cnt[0], T.cnt[1], t))) / 1000), T.cnt[0] + 0.05, T.cnt[1] - 0.02, 0.085, 0.55, 0.5);
  k.play('tock', T.cnt[1], { f: 520, gain: 0.35, pan: 0.5 }); bwoom(T.cnt[1], 0.35, 0.9, 70);
```

```
    // ---- beat 3: guess, check, adjust
    glide(5.8, 0.85, 1.0, 0, false);                                  // camera swings round to the top
view
    k.noise(T.inCard, { dur: 0.5, color: 'pink', filter: 'bandpass', f: [[0, 600], [0.5, 1800]], q: 0.8,
gain: 0.07, a: 0.4, pan: -0.5, minor: true });
    click(T.inArrow[1], 0.7, -0.3);
    k.noise(T.fwd1[0], { dur: 0.45, color: 'pink', filter: 'bandpass', f: [[0, 500], [0.45, 2200]], q:
1.2, gain: 0.08, a: 0.3, minor: true });
    click(T.outArrow[1], 0.7, 0.3);
    bwoom(T.dog, 0.5, 0.9, 72); click(T.dog, 0.9, 0.4, false);              // "dog?"
    click(when((t) => sm(T.dog + 0.05, T.dog + 0.75, t), 0.6, T.dog, T.dog + 0.8), 0.6, 0.4);
    click(T.check, 0.8, -0.5);                                        // tag turns to "cat"
    click(when((t) => sm(T.check, T.check + 0.7, t), 0.6, T.check, T.check + 0.8), 0.6, -0.5);
    k.noise(T.strike[0], { dur: 0.26, filter: 'bandpass', f: [[0, 900], [0.26, 2600]], q: 0.9, gain:
0.13, a: 0.08, pan: 0.4 });   // red strike
    k.tone(T.xmark, { f: 'E3', type: 'triangle', dur: 0.22, gain: 0.2, hold: 0.1, filter: 'lowpass', f2:
900, pan: 0.45 });         // muted "wrong"
    k.tone(T.xmark + 0.14, { f: 'C3', type: 'triangle', dur: 0.45, gain: 0.2, hold: 0.15, filter:
'lowpass', f2: 800, pan: 0.45 });
    bwoom(T.xmark, 0.45, 0.6, 50);
    k.tone(T.err[0], { f: [[0, 520], [0.45, 140]], dur: 0.5, gain: 0.07, a: 0.05, pan: 0.2 });
// error signal travels back
    k.noise(T.err[0], { dur: 0.48, color: 'pink', filter: 'bandpass', f: [[0, 1800], [0.48, 400]], q: 1,
gain: 0.1, a: 0.1, pan: 0.2 });
    click(T.err[0] + 0.3, 0.5, 0.3);
    [3, 2, 1, 0].forEach((li, i) => click(T.back[0] + (1 - (li + 0.4) / 3.8) * (T.back[1] - T.back[0]),
0.75, 0.3 - i * 0.2));       // layers light up backwards
    k.noise(T.back[0], { dur: 0.6, color: 'brown', filter: 'lowpass', f: [[0, 250], [0.6, 900]], gain:
0.12, a: 0.3, minor: true });
    click(when((t) => sm(T.adjust, T.adjust + 0.75, t), 0.6, T.adjust, T.adjust + 0.8), 0.6, 0.1);
    k.noise(T.fwd2[0], { dur: 0.3, color: 'pink', filter: 'bandpass', f: [[0, 500], [0.3, 2200]], q: 1.2,
gain: 0.08, a: 0.2, minor: true });
    chord(T.cat, ['D4', 'F#4', 'A4'], 1.1, 0.9); bwoom(T.cat, 0.45, 0.9, 74);   // "cat" correct
    // loop ring: one smooth circular pass, panned around
    [[-0.6, 0], [0, 0.2], [0.6, 0.4]].forEach(([pan, dt]) => k.noise(T.loop[0] + dt, { dur: 0.35,
color: 'pink', filter: 'bandpass', f: [[0, 400], [0.18, 1300], [0.35, 500]], q: 0.9, gain: 0.12, a: 0.15,
pan, minor: true }));
    click(T.loop[0] + 0.2, 0.6, 0.4);


    // ---- beat 4: accuracy
    glide(10.25, 0.85, 1.0, 0, false);
    click(T.acc + 0.15, 0.6, 0.2);
    bwoom(T.num[0] - 0.15, 0.35, 0.9, 66);
    ticks((t) => Math.round(accVal(eo(sm(T.num[0], T.num[1], t)))), T.num[0] + 0.02, T.num[1]
- 0.02, 0.085, 0.6, 0.3);
    air(T.num[0] + 0.2, T.lock - T.num[0] - 0.2, 1.2);
    bwoom(T.lock, 1.35, 2.2, 52);                                     // the 97% lock: biggest hit
```

```
    chord(T.lock, ['D3', 'A3', 'D4', 'F#4', 'A4'], 1.8, 1.0);
    k.noise(T.lock, { dur: 0.9, color: 'pink', filter: 'lowpass', f: [[0, 2000], [0.9, 200]], gain: 0.16, a: 0.01, verb: 0.4 });
    k.noise(T.gleam[0], { dur: 0.5, color: 'pink', filter: 'bandpass', f: [[0, 1400], [0.25, 2600], [0.5, 1500]], q: 1.5, gain: 0.05, a: 0.2, minor: true });

    // ---- beat 5: light sweep to the white stage, tagline, resolve
    air(12.75, 0.3, 0.6);
    k.noise(T.sweep[0], { dur: 0.5, color: 'pink', filter: 'bandpass', f: [[0, 380], [0.25, 3000], [0.5, 900]], q: 0.8, gain: 0.26, a: 0.22, pan: -0.4, verb: 0.3 });
    k.noise(T.sweep[0] + 0.12, { dur: 0.4, color: 'pink', filter: 'bandpass', f: [[0, 600], [0.2, 2600], [0.4, 900]], q: 0.8, gain: 0.18, a: 0.15, pan: 0.5, verb: 0.3 });
    bwoom(T.sweep[0] + 0.25, 0.55, 1.2, 60);
    T.tag.forEach((t, i) => click(t, 0.7, (i - 2) * 0.15));
    chord(T.tag[4], ['D2', 'A2', 'D3', 'F#3', 'A3', 'E4'], 1.45, 1.0);   // the resolve under the tagline
    bwoom(T.tag[4], 0.6, 1.4, 55);
    click(14.5, 0.5);
  },
 });
}
```